



A hands-on introduction to Relational Database and SQL with Postgres

Marc Linster

September 2025

Lesson Plan

- **14:00 - 17:00**
- **Start with review of prior day**
- **End with time to try things out and ask questions**
- **Hands on**

Monday	● Quick intro to databases ● Connecting to the Postgres database w. pgAdmin ● Tables and queries:Basics ● Query basics: Insert, update, delete, select ● <i>Time to exercise and ask questions</i>
Tuesday	● <i>Review of prior session and Q&A</i> ● Operators ● Primary Keys, Constraints and Relationships ● Organizing data with schemas and entity-relationship diagrams ● Data Normalization ● <i>Time to exercise and ask questions</i>
Wednesday	● <i>Review of prior session and Q&A</i> ● Queries revisited (joins, views and aggregates) ● Indexes ● A simple project ● Resources ● <i>Time to exercise and ask questions</i>



Intro and Advanced SQL Classes

Practical Introduction to SQL with Postgres

Audience: Beginners

Tool: Graphical UI (pgAdmin)

Duration: 3 days @ 3 hours

Goal:

- Familiarize with SQL and relational databases
- Write simple queries
- Read and understand data models

Advanced SQL with Postgres

Audience: Prior hands-on experience with database or programming tools

Tool: Command line (psql) & GUI (pgAdmin)

Duration: 5 days @ 3 hours

Goal:

- Read and write complex queries
- Understand transactions, indexes, and analytics concepts
- Develop normalized data models
- Develop stored procedures

Day 1

- Connect to the VMs
- Intro to databases
- Connecting to a database
- Tables and queries
- Primary keys, foreign keys, and data integrity

Connecting to the DLH Course VMs

Quick Intro to Databases



What is a database?

A **database** is an organized collection of structured data, typically stored electronically in a computer system.

Use Case Examples

- All orders that were taken, shipped, invoiced and paid in a store or eCommerce applications
- List of all laws that are currently in place
- Register of all the inhabitants of a country with their age, marital status, and domicile
- Collections of documents, X-ray images, chat conversations, and emails
- The WWW
- Data can include numbers, text, geo-locations, images, videos, links, vectors, lists, ...



Key Functions of a Database

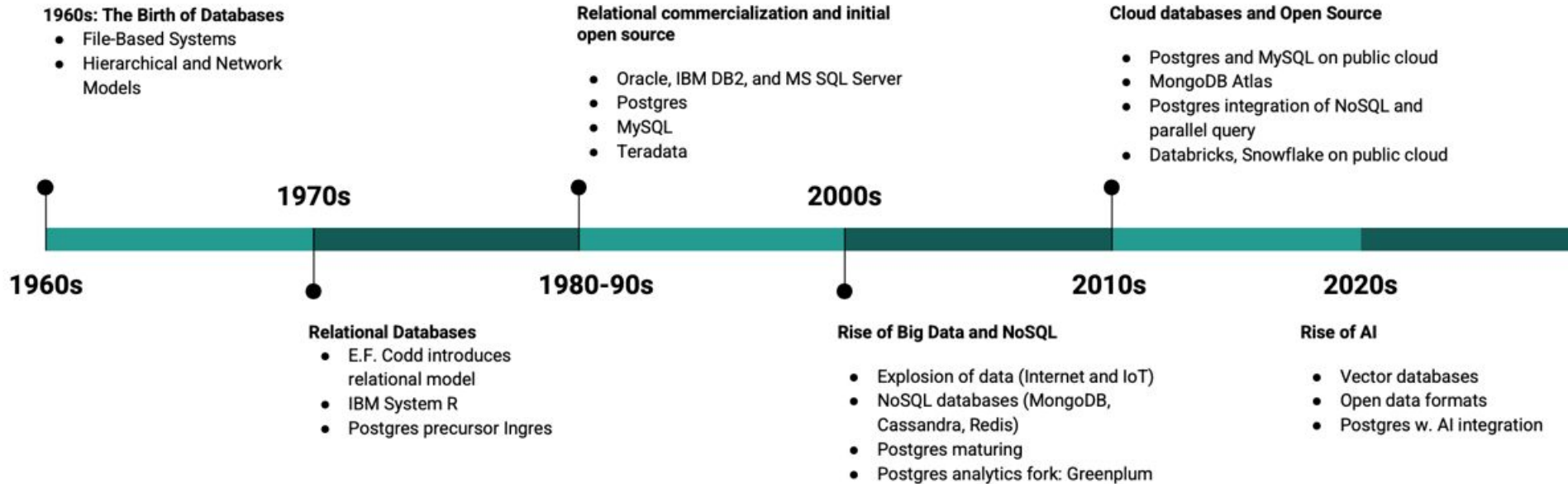
- **Data Storage:** Securely store large volumes of data.
- **Data Retrieval:** Quickly fetch specific data using queries.
- **Data Manipulation:** Insert, update, or delete data as needed.
- **Data Integrity:** Ensure accuracy and consistency through constraints and rules.
- **Data Security:** Protect data from unauthorized access with authentication and encryption.
- **Data Analysis:** Support complex queries and reporting for decision-making.



Components of a Database

- **Data:** The actual information stored, such as customer records, product details, or transaction logs.
- **Database Management System (DBMS):** Software that interacts with the database, enabling users to define, create, query, update, and manage data. Examples include MySQL, PostgreSQL, Oracle, and MongoDB.
- **Schema:** The structure or blueprint of the database, defining how data is organized (e.g., tables, fields, relationships).
- **Query Language:** A language used to interact with the database, such as SQL (Structured Query Language) for relational databases.

History of Databases

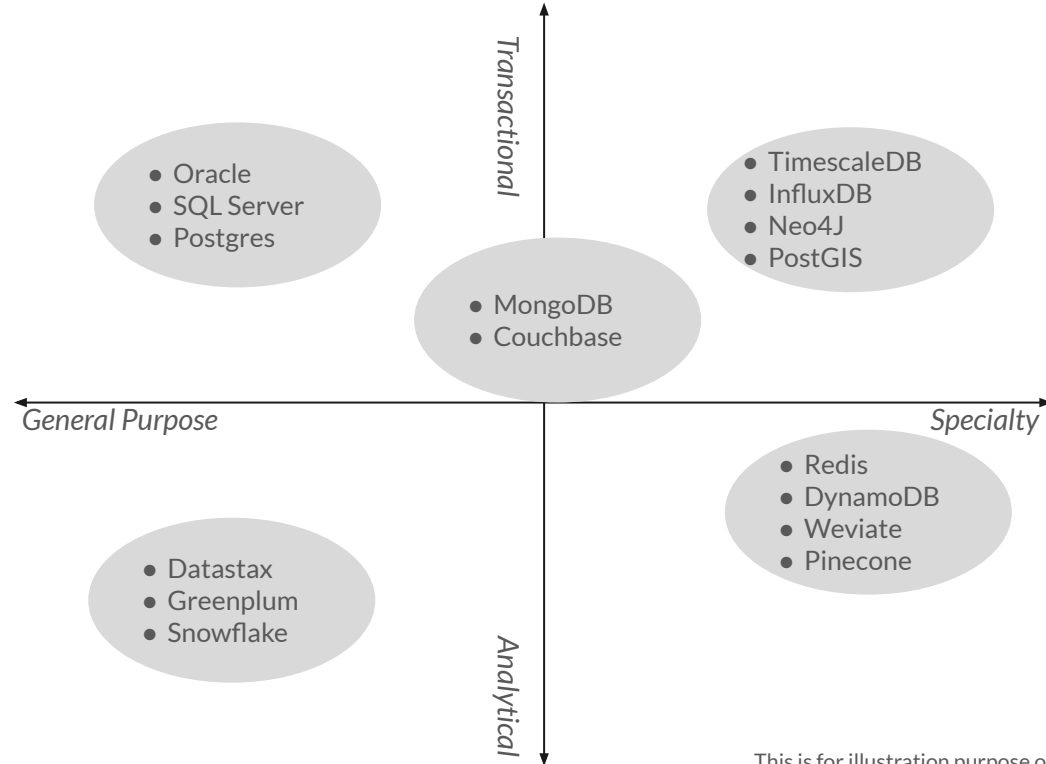




Principal Types of Databases

- There are many, many types of databases
- Key database types (not exhaustive)
 - Relational $\Leftarrow$ Our focus this week (Oracle, My SQL, Postgres, SQL Server, IBM DB2, Sybase, ...)
 - Document (MongoDB, Couchbase, ...)
 - Key Value Pair (Redis, RocksDB, Amazon Dynamo, ...)
 - Graph (Neo4J)
 - AI and vector (Pinecone, Weviate, ...)
- Features are merging!
 - Relational databases are adopting graph, document, KVP, and AI vector capabilities

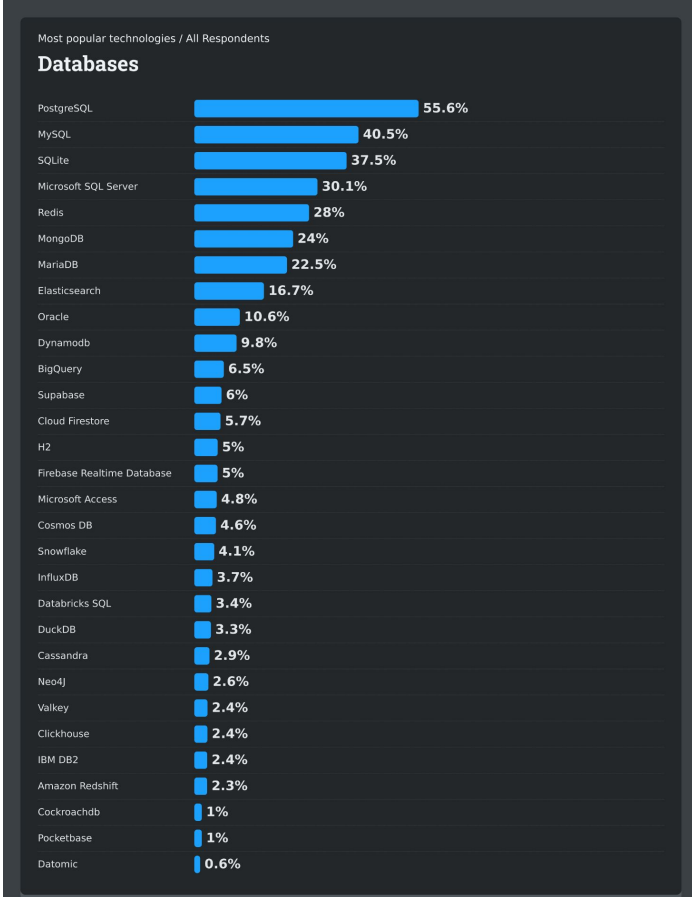
Database Landscape



This is for illustration purpose only
There are many, many databases and many ways to classify them

Stackoverflow Developer Survey 2025

Year	Rank	Developers using Postgres	Key Competitors
2025	#1	55.6%	MySQL (40.5%), SQLite (37.5%)
2024	#1	51%	MySQL (45%), SQLite (34%)
2023	#1	47%	MySQL (47%), SQLite (32%)
2022	#2	46%	MySQL (47%), SQLite (32%)
2021	#2	40%	MySQL (51%), SQLite (32%)
2020	#2	38%	MySQL (55%), SQLite (32%)
2019	#2	34%	MySQL (58%), SQLite (32%)
2018	#3	32%	MySQL (58%), SQL Server (36%)





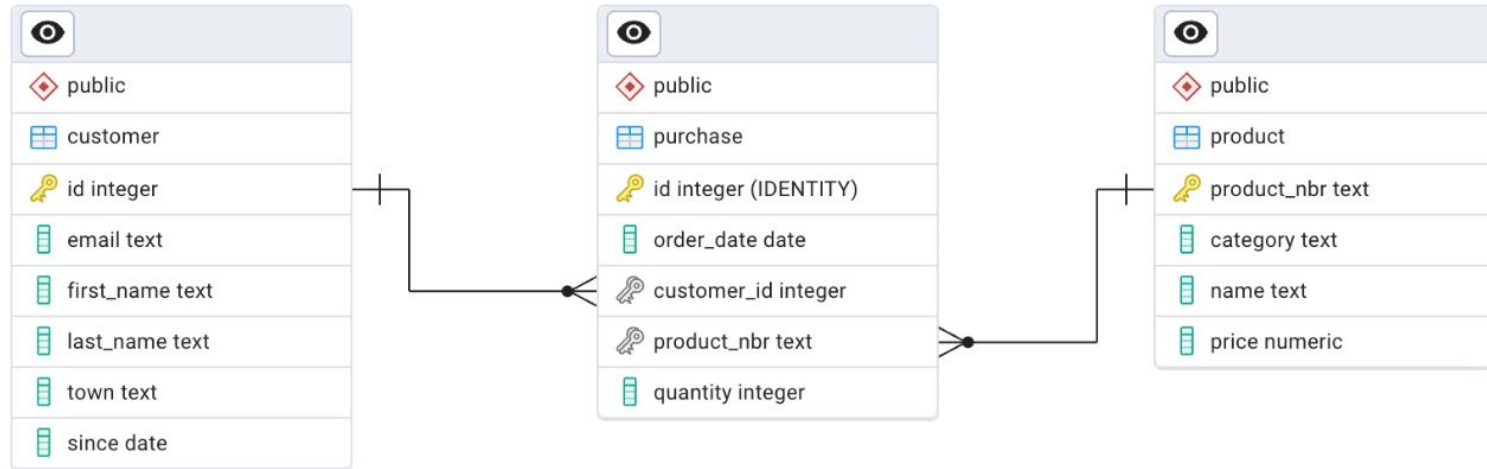
Relational Databases

- Tables, e.g., products, customers
- Relations, e.g., which customer bought which product
- Schemas, e.g., structured collections of tables and relations
- Data types, e.g., integer, numeric, float, text, time, date, ...
- Data Definition Language (DDL), e.g. CREATE TABLE, CREATE INDEX,
- Data Manipulation Language (DML), e.g., SELECT name FROM product, INSERT (first_name, last_name) VALUES ('marc', 'linster') INTO customer

 Relational Databases	Product Details		Sales Data					
	id	timestamp	product_id	customer_id	store_id	qty	unit_price	extended_price
	80	2025-12-26 6:41:08	1405	279	1	61	208.23	12702.03
	8	2025-12-07 7:20:22	392	75	4	71	203.4	14441.4
	61	2025-12-08 22:50:43	1476	237	2	27	155.91	4209.57
	59	2025-12-17 11:45:05	2232	331	3	6	132.73	796.38
	10	2025-12-11 19:56:24	113	33	4	23	153.06	3520.38
	66	2025-12-22 9:03:43	2178	70	1	61	125.83	7675.63
	51	2025-12-07 21:30:30	2082	485	2	61	138.98	8477.78
	52	2025-12-03 16:47:22	1105	443	3	69	49.7	3429.3
Customer Information		Order History						
customer_id	name	email	phone	address	city	state	zip	country
101	John Doe	john.doe@example.com	555-123-4567	123 Main St	New York	NY	10001	USA
102	Jane Smith	jane.smith@example.com	555-987-6543	456 Elm St	Los Angeles	CA	90001	USA
103	Bob Johnson	bob.johnson@example.com	555-234-5678	789 Oak St	Chicago	IL	60601	USA
104	Alice Brown	alice.brown@example.com	555-345-6789	101 Pine St	San Francisco	CA	94101	USA
105	Charlie Davis	charlie.davis@example.com	555-456-7890	202 Maple St	Phoenix	AZ	85001	USA
106	Diana Prince	diana.prince@example.com	555-567-8901	303 Cedar St	Philadelphia	PA	19101	USA
107	Frank Miller	frank.miller@example.com	555-678-9012	404 Birch St	San Diego	CA	92101	USA
108	Grace Wilson	grace.wilson@example.com	555-789-0123	505 Walnut St	Portland	OR	97201	USA
109	Henry Taylor	henry.taylor@example.com	555-890-1234	606 Spruce St	Seattle	WA	98101	USA
110	Ivy Green	ivy.green@example.com	555-901-2345	707 Ash St	Denver	CO	80201	USA

name		description		manufacturer_id	size	color	sku	price
Cotton T-Shirt		Handcrafted details		3	XXL	Light Green	FSH-89166-000000000165	11.4
Wool Socks		Trendy and fashionable		1	M	Dark Yellow	FSH-53451-000000000138	146.26
Wool Dress Pants		Elegant and stylish		2	S	Turquoise	FSH-25089-0000000001213	102.79
Evening Gown		Waterproof and windproof		3	XS	Light Green	FSH-47771-000000000115	103.36
Wool Dress Pants		Elegant and stylish		2	XXL	Dark Blue	FSH-13141-000000000164	16.41
Baseball Cap		Versatile and functional		3	XL	Light Red	FSH-43094-000000000151	160.5
Slacks		Trendy and fashionable		3	XL	Dark Blue	FSH-82072-000000000154	39.63
Leather Jacket		Comfortable and durable		3	XS	Amber	FSH-15582-0000000001114	172.11
Sunglasses		Quick-drying		2	L	Dark Blue	FSH-65791-000000000144	18.73
Leather Belt		Comfortable and durable		3	XXL	Amber	FSH-11391-0000000001614	62.81

Entity-Relationship Diagrams





Document Database

- Documents encapsulate and encode data, e.g., customer
- Collections gather similar documents, e.g., customers
- No standard schema, e.g., some customer documents can have fields that differ. For example
 - Customer1: firstname, lastname, address_line_1, city, postalcode
 - Customer2: firstname, middlename, lastname, address_line_1, address line 2, city, postalcode



Document Database

```
{
  "customer_id": "12345",
  "name": "John Doe",
  "email": "johndoe@example.com",
  "phone": "+1234567890",
  "address": {
    "street": "123 Main St",
    "city": "Anytown",
    "state": "CA",
    "zip": "90210"
  },
}
```

```
"orders": [
  {
    "order_id": "67890",
    "customer_id": "12345",
    "date": "2023-10-01",
    "total_amount": 150.75,
    "items": [
      {
        "product_id": "p123",
        "product_name": "Wireless
Mouse",
        "quantity": 1,
        "price": 50.25
      },
      {
        "product_id": "p456",
        "product_name": "Bluetooth
Keyboard",
        "quantity": 1,
        "price": 100.50
      }
    ]
  },
]
```



Key Value Pair Database

- 'Values' associated with a 'Key'
 - `Customer: 123 name "Marc Linster" city "Junglinster"`
 - `Product:xyz price "192,17" color"blue"`
- Extremely fast access to data
- Usually stored in memory
- Used as cache or working memory for applications



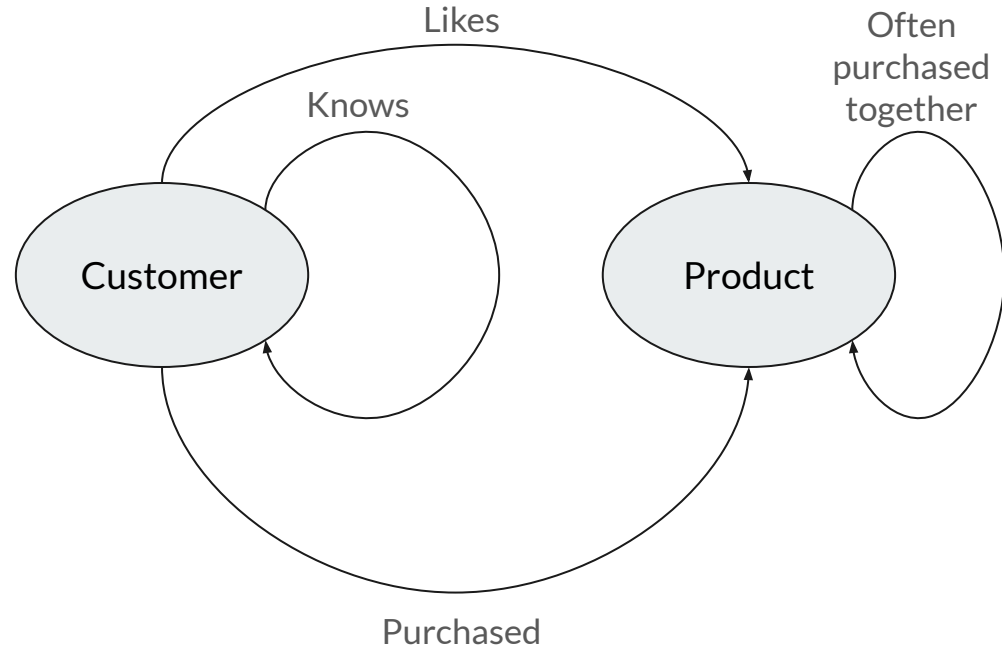
Key Value Pair Database

```
HSET customer:12345 name "John Doe" email "johndoe@example.com" phone "+1234567890"  
HSET customer:12345 address '{"street": "123 Main St", "city": "Anytown", "state": "CA", "zip": "90210"}'  
  
HSET customer:67890 name "Jane Smith" email "janesmith@example.com" phone "+0987654321"  
HSET customer:67890 address '{"street": "456 Elm St", "city": "Othertown", "state": "NY", "zip": "10001"}'
```

```
HSET order:67890 customer_id "12345" date "2023-10-01" total_amount "150.75"  
HSET order:67890 items '[{"product_id": "p123", "product_name": "Wireless Mouse", "quantity": 1, "price":  
50.25}, {"product_id": "p456", "product_name": "Bluetooth Keyboard", "quantity": 1, "price": 100.50}]'  
  
HSET order:67891 customer_id "12345" date "2023-10-05" total_amount "75.00"  
HSET order:67891 items '[{"product_id": "p789", "product_name": "USB-C Hub", "quantity": 2, "price":  
37.50}]'
```

Graph Database

- Nodes, e.g., customer, product
- Relationships, e.g., purchased, likes, knows





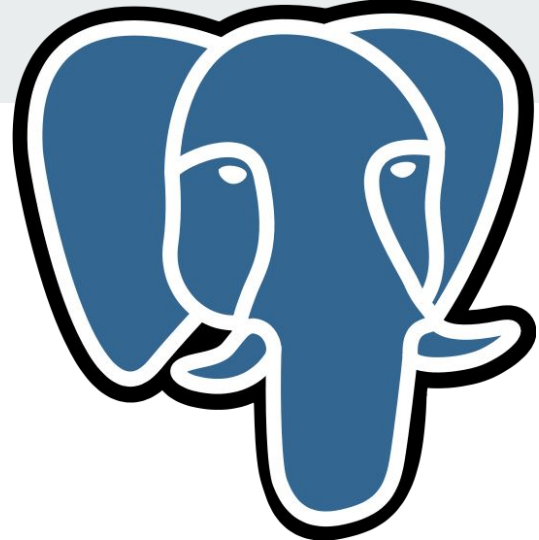
Graph Database

```
CREATE (c1:Customer {id: 1, name: "John Doe", email: "johndoe@example.com"});  
CREATE (c2:Customer {id: 2, name: "Jane Smith", email: "janesmith@example.com"});  
CREATE (c3:Customer {id: 3, name: "Alice Johnson", email: "alicej@example.com"});
```

```
CREATE (p1:Product {id: 101, name: "Wireless Mouse", price: 25.99});  
CREATE (p2:Product {id: 102, name: "Bluetooth Keyboard", price: 49.99});  
CREATE (p3:Product {id: 103, name: "USB-C Hub", price: 35.50});  
CREATE (p4:Product {id: 104, name: "Monitor", price: 199.99});
```

```
MATCH (c1:Customer {id: 1}), (p1:Product {id: 101}) CREATE (c1)-[:LIKES]->(p1);  
MATCH (c1:Customer {id: 1}), (p2:Product {id: 102}) CREATE (c1)-[:LIKES]->(p2);  
MATCH (c2:Customer {id: 2}), (p3:Product {id: 103}) CREATE (c2)-[:LIKES]->(p3);  
MATCH (c3:Customer {id: 3}), (p4:Product {id: 104}) CREATE (c3)-[:LIKES]->(p4);
```

Connecting to a Postgres Database



Tools for this course

Editor

- pgAdmin 9.6
- License: Postgres License

Postgres

- PostgreSQL V17.6
- License: Postgres License

Exercises and Samples

- License: Creative Commons (no Rights Reserved)

It's all 100% Open Source

Take it home, use it,
learn from it

Basic elements of a Postgres database connection from pgAdmin

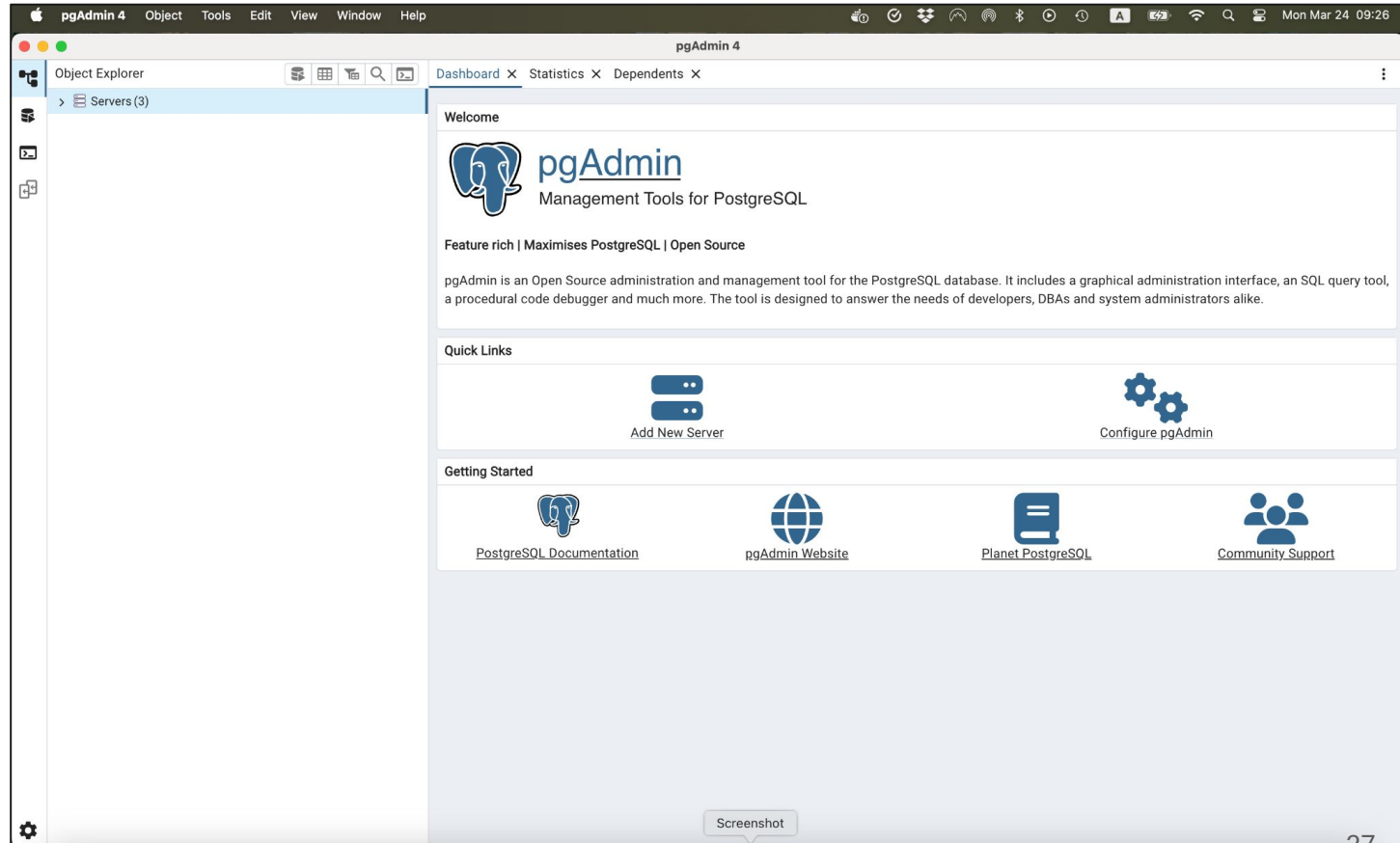
Connection name	The name in the GUI	• dlhtraining ← • My connection 1
Where is the database server?	Network address (IP address and port)	IP Address • Localhost ← • 188.115.7.175 • mydb.aws.com Port • 5432 ← • 80
Which database do I connect to?	Name of the database on the servers	• DLH1 ← • DLH2 • DLH3 • inventory
Who am I?	Username and password	Username • postgres ← • dhlstudent Password • postgres ← • xd67\$566
Security parameters	Encryption on the network	• Not applicable in training



Connection elements

- Network address
 - IP V4 example: 188.115.7.175
 - IP V6 example: 2001:7e8:c415:f601:c0fe:c98b:257a:a45b
 - url: mydb.post.lu
 - Special address: **localhost** (always the local computer. In IPV4: "127.0.0.1", in IPV6: ":::1")
- (Network) Port:
 - The port defines which service at a specific network address I want to use, and what 'language' (protocol) I can speak at that port
 - Examples:
 - Port 20: SMTP (email)
 - Port 80: HTTP (WWW)
 - **Port 5432: Postgres**

pgAdmin



pgAdmin 4

ObjectToolsEditViewWindowHelp

DLH1

pgAdmin 4

Dashboard XStatistics XDependents X

Object Explorer

Servers

Register

Create

Refresh...

Remove Server Group

Disconnect from all servers

Properties...

Server...

Deploy Cloud Instance...

pgAdmin

Management Tools for PostgreSQL

Feature rich | Maximises PostgreSQL | Open Source

pgAdmin is an Open Source administration and management tool for the PostgreSQL database. It includes a graphical administration interface, an SQL query tool, a procedural code debugger and much more. The tool is designed to answer the needs of developers, DBAs and system administrators alike.

Quick Links

Add New Server

Configure pgAdmin

Getting Started

PostgreSQL Documentation

pgAdmin Website

Planet PostgreSQL

Community Support

Servers

- Servers (2)
 - dlh1
 - postgres17
 - Databases (4)
 - dlh1
 - Catalogs
 - Extensions
 - Languages
 - Schemas (1)
 - public
 - 1..3 Sequences
 - Tables (3)
 - mycustomer
 - myorder
 - myproduct
 - dlh2
 - dlh3
 - postgres
 - Login/Group Roles
 - Tablespaces

dlh3/postgres@postgres17



Query Query History

1

Data Output Messages Notifications



Object Explorer



Dashboard X Statistics X Dependents X



Servers (2)

> dhl1

dhl1_student

Databases (1)

postgres

> Casts

> Catalogs

> Event Triggers

> Extensions

> Foreign Data Wrappers

> Languages

> Publications

Schemas (1)

public

> Aggregates

> Collations

> Domains

> FTS Configurations

> FTS Dictionaries

> FTS Parsers

> FTS Templates

> Foreign Tables

> Functions

> Materialized Views

> Operators

> Procedures

> 1.3 Sequences

Tables (6)

> address

> customer

> manufacturer

> product

> sales_transaction

> store

Please connect to the selected server to view the dashboard.



Object Explorer

- Servers (2)
 - dlh1
 - postgres17
 - Databases (4)
 - dlh1
 - Catalogs
 - Extensions
 - Languages
 - Schemas (1)
 - public
 - Sequences
 - Tables (3)
 - mycustomer
 - Columns (5)
 - id
 - email
 - first_name
 - last_name
 - since
 - Constraints
 - myorder
 - myproduct
 - dlh2
 - dlh3
 - postgres
 - Login/Group Roles
 - Tablespaces

public.mycustomer/dlh1/postgres@postgres17

100 rows

Query Query History

```
1 SELECT * FROM public.mycustomer
2 LIMIT 100
3
```

Data Output Messages Notifications



Showing rows: 1 to 3 Page No: 1 of 1

| | id | email | first_name | last_name | since |
|---|---------|-----------------------|------------|-----------|-------------|
| | integer | text | text | text | date |
| 1 | 1 | marc@marclinster.c... | Marc | Linster | 2025-01-... |
| 2 | 2 | jmu@gmail.lu | Jeff | Mueller | 2025-01-... |
| 3 | 3 | bini@hotmail.lu | Jean | Bintner | 2025-01-... |

▼

▼

▼

▼

100 rows

▼

▼

E

||

▼

1

2

3

▼

?

Query Query History

```
1 SELECT * FROM public.customer
2 LIMIT 100
3
```

Data Output Messages Notifications

Showing rows: 1 to 100 Page No: 1 of 1

| | id
integer | customer_number
character varying (25) | first_name
character varying (25) | last_name
character varying (25) | address_id
integer | phone_numbers
jsonb |
|----|---------------|---|--------------------------------------|-------------------------------------|-----------------------|---|
| 1 | 1 | cust00001 | Ella | Walter | 80900 | {\"home\": \"+352 32 07 30\"} |
| 2 | 2 | cust00002 | Enzo | Rutkowski | 137013 | {\"home\": \"+352 621 375 964\"} |
| 3 | 3 | cust00003 | Clara | Arnold | 26711 | {\"home\": \"+352 621 159 268\", \"work\": \"+352 79 23 31\", \"mobile\": \"+352 621 159 268\"} |
| 4 | 4 | cust00004 | Dylan | Ziółkowska | 97658 | {\"home\": \"+352 62 09 85\", \"work\": \"+352 621 900 166\", \"mobile\": \"+352 621 900 166\"} |
| 5 | 5 | cust00005 | Felix | Kaczmarczyk | 132220 | {\"home\": \"+352 77 24 31\"} |
| 6 | 6 | cust00006 | Emma | Mazurek | 125998 | {\"home\": \"+352 57 64 83\"} |
| 7 | 7 | cust00007 | Enzo | Kaczmarczyk | 15193 | {\"home\": \"+352 82 69 48\", \"work\": \"+352 621 707 758\", \"mobile\": \"+352 621 707 758\"} |
| 8 | 8 | cust00008 | Victoria | Urbański | 175317 | {\"home\": \"+352 621 929 105\"} |
| 9 | 9 | cust00009 | Anna | Fischer | 43145 | {\"home\": \"+352 621 665 543\"} |
| 10 | 10 | cust00010 | Lena | Lange | 137847 | {\"home\": \"+352 20 15 75\", \"work\": \"+352 621 363 703\", \"mobile\": \"+352 621 363 703\"} |
| 11 | 11 | cust00011 | Nina | Wróblewski | 102195 | {\"home\": \"+352 65 71 21\"} |
| 12 | 12 | cust00012 | Leonor | Hübner | 147959 | {\"home\": \"+352 25 82 89\", \"work\": \"+352 621 363 703\", \"mobile\": \"+352 621 363 703\"} |

✓ Table rows counted: 500 ✗

Tables and Queries: Basics

Table Basics

- Rows = records (a.k.a. tuples in Postgres)
- Columns = data elements
- Example: mycustomer table
 - ID = a way to uniquely identify a customer $\Rightarrow$ usually a large number
 - Email = the email address, a text field, sometimes limited in length
 - First name $\Rightarrow$ a text field, sometimes limited in length
 - Last name $\Rightarrow$ a text field, sometimes limited in length
 - Since $\Rightarrow$ a date field

```
postgres=# SELECT * FROM mycustomer;
```

| id | email | first_name | last_name | since |
|----|----------------------|------------|-----------|------------|
| 1 | marc@marclinster.com | Marc | Linster | 2025-01-01 |
| 2 | jmu@gmail.lu | Jeff | Mueller | 2025-01-02 |
| 3 | bini@hotmail.lu | Jean | Bintner | 2025-01-03 |



Tables

- All rows/records have the same columns,
 - E.g., for every customer we have id, first_name, last_name, since
- Every column has a data type, which constrains what data can be in that position
 - E.g., 'since' is always a date, first_name is always a text
- Most important data types
 - Boolean: True/False
 - Character types: char, varchar, text
 - Numeric: integer, floating point, ...
 - Temporal types: date, time, timestamp, interval
 - UUID: Unique identifies
 - Array: arrays of strings, numbers, etc
 - JSON: structured documents
 - Special types for network addresses, XML, geographic points, etc



Illustrating Select Postgres Data Types

Boolean: 'true' or 'false'

Character Types

- Fixed length: CHARACTER(n), CHAR(n)
- Variable length w. length limit: CHARACTER VARYING (n), VARCHAR (n)
- Variable w.o. length limit: TEXT, VARCHAR

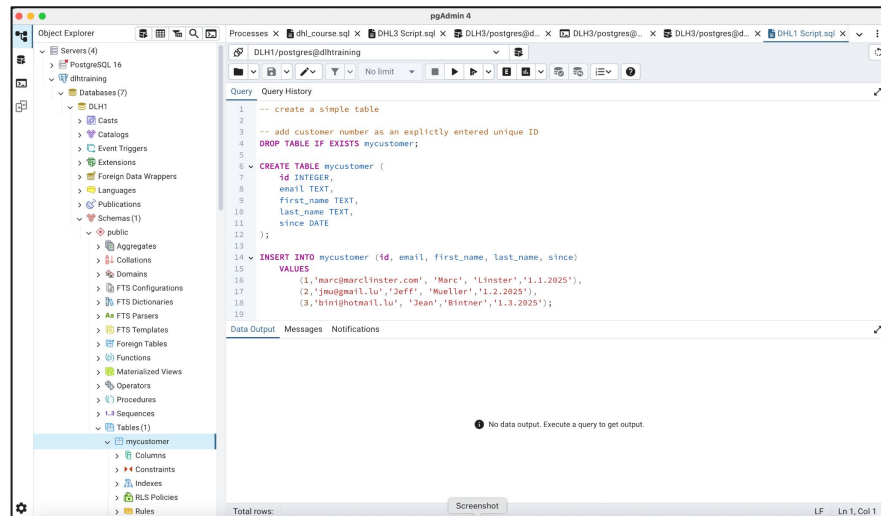
Number Types

- INTEGER:
 - SMALLINT (+/- 32,768), INTEGER (+/- 2,147,483,648), BIGINT (+/- 9,223,372,036,854,775,808)
- NUMERIC/DECIMAL:
 - NUMERIC (5,2) - 5 digits before the period, two after the period, for example 172.99

Let's try it out

- pgAdmin Query Tool
 - Make sure you are connected to DLH1!
 - Enter `SELECT current_database();` to check where you connected
 - Try this:


```
CREATE TABLE mycustomer (
    id INTEGER,
    email TEXT,
    first_name TEXT,
    last_name TEXT,
    since DATE
);
```
 - Check out the list of tables in the pgAdmin tree browser (hit refresh)



Uppercase, lowercase, white spaces, etc

- Postgres ignores case - all names for tables, columns, functions, procedures etc. are lowercase
 - Postgres automatically lower cases all identifiers!!!
 - SQL identifiers and keywords must begin with a letter or an underscore (_).
 - Subsequent characters can be letters, underscores, digits (0-9), or dollar signs (\$).
 - Note that dollar signs are not allowed in identifiers according to the letter of the SQL standard, so their use might render applications less portable.
- Customername = CustomerName = CUSTOMERNAME = customername <> "CustomerName" <> "customerName" <> "CUSTOMERNAME"
- Do not use camelCase, use snake case to improve readability of identifiers, e.g. customer_name
- Spaces (' ') cannot be used in identifiers
- Double quotes "" can be used to create mixed case identifiers that include spaces

DON'T DO IT!!!

Mixed case and spaces cause all kinds of problems later on

Query Basics



Query types

- SELECT: List data from tables that meet certain conditions
- UPDATE: Change data in records that meet certain conditions
- INSERT: Add records
- DELETE: Remove Records

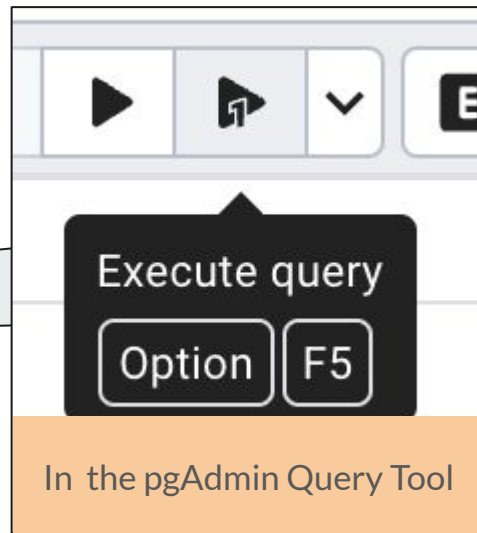
Let's try it out

```
SELECT * FROM mycustomer;
```

- Remember the ';
- Click

```
INSERT INTO mycustomer (id, email, first_name, last_name, since)
VALUES
    (1, 'marc@marclinster.com', 'Marc', 'Linster', '1.1.2025'),
    (2, 'jmu@gmail.lu', 'Jeff', 'Mueller', '1.2.2025'),
    (3, 'bini@hotmail.lu', 'Jean', 'Bintner', '1.3.2025');
```

```
SELECT first_name FROM mycustomer;
SELECT first_name, last_name FROM mycustomer;
SELECT * FROM customer WHERE last_name = 'Linster';
SELECT * FROM customer WHERE first_name LIKE 'Je%';
```





Building Blocks of the simple SELECT Query

- **SELECT**
 - Columns to be selected, maybe using AS to provide a meaningful name
 - For example `last_name AS ln`
- **FROM**
 - Table(s) from which to select. Can also use AS to provide a new name
 - For example `customer AS c`
- **WHERE**
 - Filter clauses to select a subset of records
 - Can use the new names from the AS clause
 - For example `c.ln LIKE 'Li%'`
- **ORDER BY**
 - Organizes the output of the SELECT clause
 - Can use the new names from the AS clause
 - For example: `c.ln ASC, since DESC`

Alias (AS)

- improves readability
- Only available in the context of this query
- Key word 'AS' is optional

More complex building blocks for SELECT queries (JOIN, GROUP BY, HAVING, GROUPING, WITH, ...) will be introduced later



Let's try it out

```
UPDATE mycustomer SET first_name = 'Mark' WHERE id = 1;
SELECT * FROM mycustomer;
DELETE FROM mycustomer WHERE id = 1
SELECT * FROM mycustomer;
SELECT * FROM mycustomer ORDER BY last_name ASC;
```

Try it in pgAdmin Query Tool

Day 2

- Review of Day 1
- Operators
- Relationships and Constraints
- Organizing data - schemas, entity relationship diagrams, and normalization
- Working with data: data normalization and avoiding redundancy

Operators



Postgres Operators in the WHERE Clause

| | |
|---------|--|
| = | Equal to |
| < | Less than |
| > | Greater than |
| <= | Less than or equal to |
| >= | Greater than or equal to |
| <> | Not equal to |
| != | Not equal to |
| LIKE | Check if a value matches a pattern (case sensitive) |
| ILIKE | Check if a value matches a pattern (case insensitive) |
| AND | Logical AND |
| OR | Logical OR |
| IN | Check if a value is between a range of values |
| BETWEEN | Check if a value is between a range of values |
| IS NULL | Check if a value is NULL |
| NOT | Makes a negative result e.g. NOT LIKE, NOT IN, NOT BETWEEN |



Let's try it out

```
SELECT * FROM mycustomer WHERE first_name LIKE 'Je%';
```

```
SELECT * FROM mycustomer WHERE first_name LIKE 'je%';
```

```
SELECT * FROM mycustomer WHERE first_name ILIKE 'je%';
```

```
SELECT * FROM mycustomer WHERE first_name LIKE '_e%';
```

```
SELECT * FROM mycustomer WHERE since  
    BETWEEN '2025-01-02' AND '2025-02-01';
```

Postgres functions in the SELECT clause

Some of the key string functions

| | | |
|------------------------|---|---|
| <code> </code> | String append | <code>'a' '_' 'b' → a_b</code> |
| <code>lpad/rpad</code> | Create a padded string | <code>rpad('hi', 5, 'xy') → hixyx</code> |
| <code>substring</code> | Create a substring | <code>substring('Thomas' from 2 for 3) → hom</code> |
| <code>::string</code> | Cast a number (and other types) as string | <code>1::TEXT '_' 2::TEXT → 1_2</code> |

Many, many more in the Postgres documentation: <https://www.postgresql.org/docs/current/functions-string.html>

More examples: EXTRACT and DAY

- EXTRACT from timestamp or date
 - CENTURY
 - DAY (day of the month)
 - DOW (day of the week - Sunday(0) to Saturday (6))
 - ISODOW (ISO standard day of the week - Monday (1) to Sunday (7))
 - DOY (day of the year)
 - HOUR, MINUTE,
- Try it out:
 - ```
SELECT 'It is the ' || EXTRACT (DOY FROM current_date) || 'th day of the year';
```

```
 ?column?
```

```

```

```
It is the 234th day of the year
```
- Name the column using AS
  - ```
SELECT 'It is the ' || EXTRACT (DOY FROM current_date) || 'th day of the year' as  
day_of_year;
```



Lets try it out

```
SELECT
    FIRST_NAME || ' ' || LAST_NAME AS customer_name
FROM
    MYCUSTOMER
WHERE
    FIRST_NAME LIKE 'Je%';
```



A better way to format strings: FORMAT

```
SELECT
    FORMAT ('%s %s', first_name, last_name)
        AS customer_name
FROM
    MYCUSTOMER
WHERE
    FIRST_NAME LIKE 'Je%';
```

Calculations in the SELECT statement

```
SELECT
    FORMAT ('%s %s', first_name, last_name)
        AS customer_name,
    'Customer for ' || EXTRACT (DAY FROM NOW() - since) || ' ' || 'days'
        AS how_long
FROM
    MYCUSTOMER;
```

| customer_name | how_long |
|---------------|-----------------------|
| Marc Linster | Customer for 111 days |
| Jeff Mueller | Customer for 110 days |
| Jean Bintner | Customer for 109 days |

EXTRACT (field FROM source)

- Field: DAY, DOW, DOY, YEAR, ...
- SOURCE: Date, timestamp or interval

See:

<https://neon.tech/postgresql/postgresql-date-functions/postgresql-extract>



Try it out

- Create a query that
 - Selects customers from the customer table
 - Puts the name in the format 'last name, first name', e.g., Linster, Marc
 - Adds a columns entitled 'customer_since'
 - Sorts descending by last name

This code is not in the sample file - please write it on your own!

Primary Keys, Constraints and Relationships



Constraints and Relationships

- Constraints and relationships make sure your data in the tables is consistent
 - For example:
 - Orders always have quantities > 0
 - Products with prices > 0
 - Products each having different names
 - Orders referring to existing products
- Constraints and relationships complement the definition of tables, columns, and data types



Constraints and Relationships

- **Check Constraints:** The value in the column has to meet certain criteria, e.g., price > 0
- **Not-Null Constraints:** The column needs to have a value
- **Unique Constraints:** No two records have the same value
- **Primary Keys:** like the Unique Constraint, but excludes NULL and always creates an index
 - ⇒ A primary key can also be a combination of columns
- **Foreign Keys:** the data needs to be a valid reference to another table
 - You can assign a 'trigger' to handle deletions



Primary Keys

- A unique way to find a row in the table
 - Cannot be NULL
 - Must be unique
- Can be a combination of rows

| id | email | first_name | last_name | since |
|----|----------------------|------------|-----------|------------|
| 1 | marc@marclinster.com | Marc | Linster | 2025-01-01 |
| 2 | jmu@gmail.lu | Jeff | Mueller | 2025-01-02 |
| 3 | bini@hotmail.lu | Jean | Bintner | 2025-01-03 |



Table Basics

```
CREATE TABLE myproduct (  
    product_nbr VARCHAR(10),  
    name TEXT,  
    price numeric  
);
```

Problems:

- Every product should have a unique id (or product number)
- No two products should have the same name
- Every product should have a price > 0

Table Basics

```
DROP TABLE IF EXISTS myproduct;
```

```
CREATE TABLE myproduct (
  nbr VARCHAR(10) PRIMARY KEY,
  name TEXT UNIQUE NOT NULL,
  price NUMERIC NOT NULL CHECK (price > 0)
);
```

Try it out

- Insert 4 product records
- Use SELECT to make sure it worked
- Sort them by price ascending

- Add primary key to uniquely identify every record
 - PRIMARY KEY = UNIQUE + NOT NULL + indexed
 - Only one primary key per table
- Make sure that no products can have the same name
- All prices have to be greater than 0

```
SELECT * FROM myproduct;
```

| nbr | name | price |
|------------|------------------------|-------|
| ham_001 | Hammer 200gr | 19.95 |
| ham_002 | Hammer 1kg | 25.00 |
| chisel_004 | Chisel stone | 39.95 |
| pli_019 | Pliers red, adjustable | 13.50 |

Relations and foreign keys

```
DROP TABLE IF EXISTS myorder;
```

```
CREATE TABLE myorder (  
    id INT PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,  
    date DATE,  
    product_nbr VARCHAR(10) NOT NULL,  
    qty INTEGER NOT NULL CHECK (qty > 0)  
)  
-- this is a nonsense entry as we don't have such a product  
INSERT INTO myorder  
    (date, product_nbr, qty)  
VALUES ('2.2.2025', 'cheese_10', 9);
```

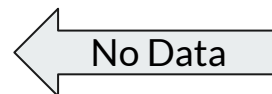
```
SELECT * FROM myorder;
```

```
SELECT * FROM myorder, myproduct  
WHERE myorder.product_nbr = myproduct.nbr;
```

- Create PRIMARY KEY
- Automatically generate a unique ID

Problems:

- No guarantee that the product_nbr is actually in the myproduct table!



Relations and foreign keys

```
DROP TABLE IF EXISTS myorder;
```

```
CREATE TABLE myorder (
  id INT PRIMARY KEY
    GENERATED BY DEFAULT AS IDENTITY,
  date DATE,
  product_nbr VARCHAR(10) REFERENCES myproduct (nbr),
  qty INTEGER NOT NULL CHECK (qty > 0));
```

create an order for a chisel

```
INSERT INTO myorder (date,product_nbr, qty)
VALUES ('2.2.2025', 'chisel_004', 9);
```

try to create an order for a non-existing product

```
INSERT INTO myorder (date,product_nbr,
qty)
VALUES ('2.2.2025', 'cheese_10', 9);
```

```
ERROR: insert or update on table
"myorder" violates foreign key constraint
"myorder_product_nbr_fkey"
Key (product_nbr)=(cheese_10) is not
present in table "myproduct".
```

```
SQL state: 23503
Detail: Key
(product_nbr)=(cheese_10) is not present
in table "myproduct".
```

```
SELECT * FROM myorder, myproducts
WHERE
  myorder.product_nbr =
  myproduct.nbr;
```

Orders and Products Connected

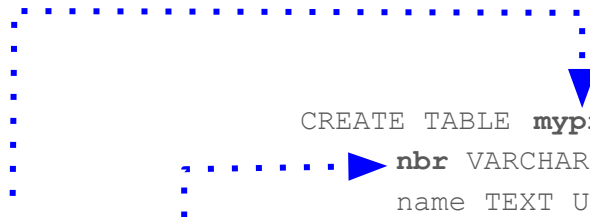
```
SELECT * FROM myorder, myproduct
WHERE myorder.product_nbr = myproduct.nbr;
```

| id | date | product_nbr | qty | nbr | name | price |
|----|------------|-------------|-----|------------|--------------|-------|
| 1 | 2025-02-02 | chisel_004 | 9 | chisel_004 | Chisel stone | 39.95 |

(1 row)

```
CREATE TABLE myorder (
  id INT PRIMARY KEY
  GENERATED BY DEFAULT AS IDENTITY,
  date DATE,
  product_nbr VARCHAR(10) REFERENCES myproduct (nbr),
  qty INTEGER NOT NULL CHECK (qty > 0));
```

```
CREATE TABLE myproduct (
  nbr VARCHAR(10) PRIMARY KEY,
  name TEXT UNIQUE NOT NULL,
  price NUMERIC NOT NULL CHECK (price > 0)
);
```



Linking customers, orders and products

- Add a column to table 'myorder' to reference the table 'mycustomer'

```
CREATE TABLE myorder (  
    id INT PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,  
    date DATE,  
    product_nbr VARCHAR(10) REFERENCES myproduct (nbr),  
    qty INTEGER NOT NULL CHECK (qty > 0)  
);
```

```
ALTER TABLE myorder ADD COLUMN customer_id INTEGER REFERENCES mycustomer(id);
```

⇒ fails because the `customer_id` is not guaranteed to be unique

⇒ Add unique constraint to table mycustomer(id)

```
ALTER TABLE mycustomer ADD CONSTRAINT mycustomer_id_unq UNIQUE (id);
```

```
ALTER TABLE myorder ADD COLUMN customer_id INTEGER REFERENCES mycustomer(id);
```

⇒ success

Linking an order with a customer

```
SELECT * FROM myorder;
```

```
postgres=# SELECT * FROM myorder;
```

| id | date | product_nbr | qty | customer_id |
|----|------------|-------------|-----|-------------|
| 1 | 2025-02-02 | chisel_004 | 9 | |

(1 row)

Foreign key constraint was added AFTER the record was inserted. Not checked!

```
UPDATE myorder SET customer_id = 1 WHERE id = 1;
```

```
SELECT * FROM myorder o,  
        myproduct p,  
        mycustomer c  
WHERE  
        o.product_nbr = p.nbr  
AND  
        o.customer_id = c.id;
```

```
-[ RECORD 1 ]-----  
id            | 1  
date          | 2025-02-02  
product_nbr   | chisel_004  
qty           | 9  
customer_id   | 1  
nbr           | chisel_004  
name          | Chisel stone  
price         | 39.95  
id            | 1  
email         | marc@marclinster.com  
first_name    | Marc  
last_name     | Linster  
since         | 2025-01-01
```

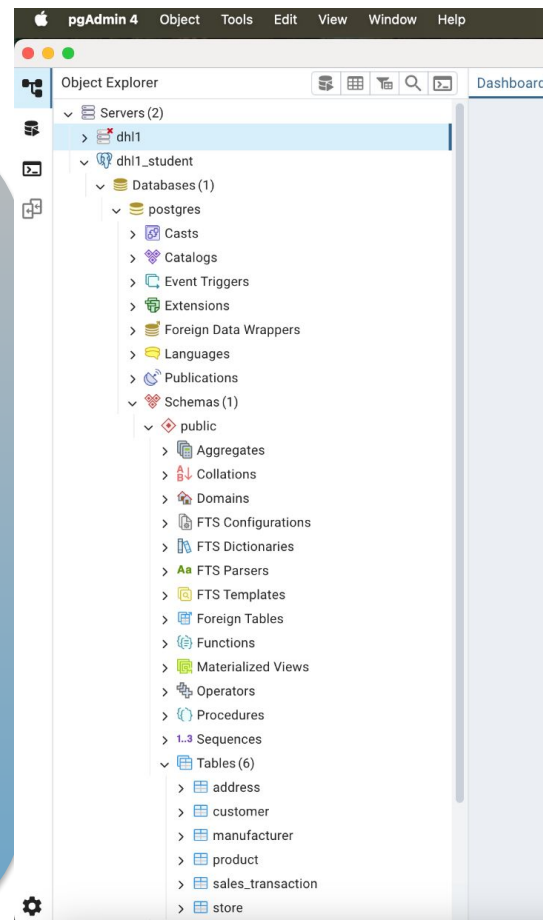
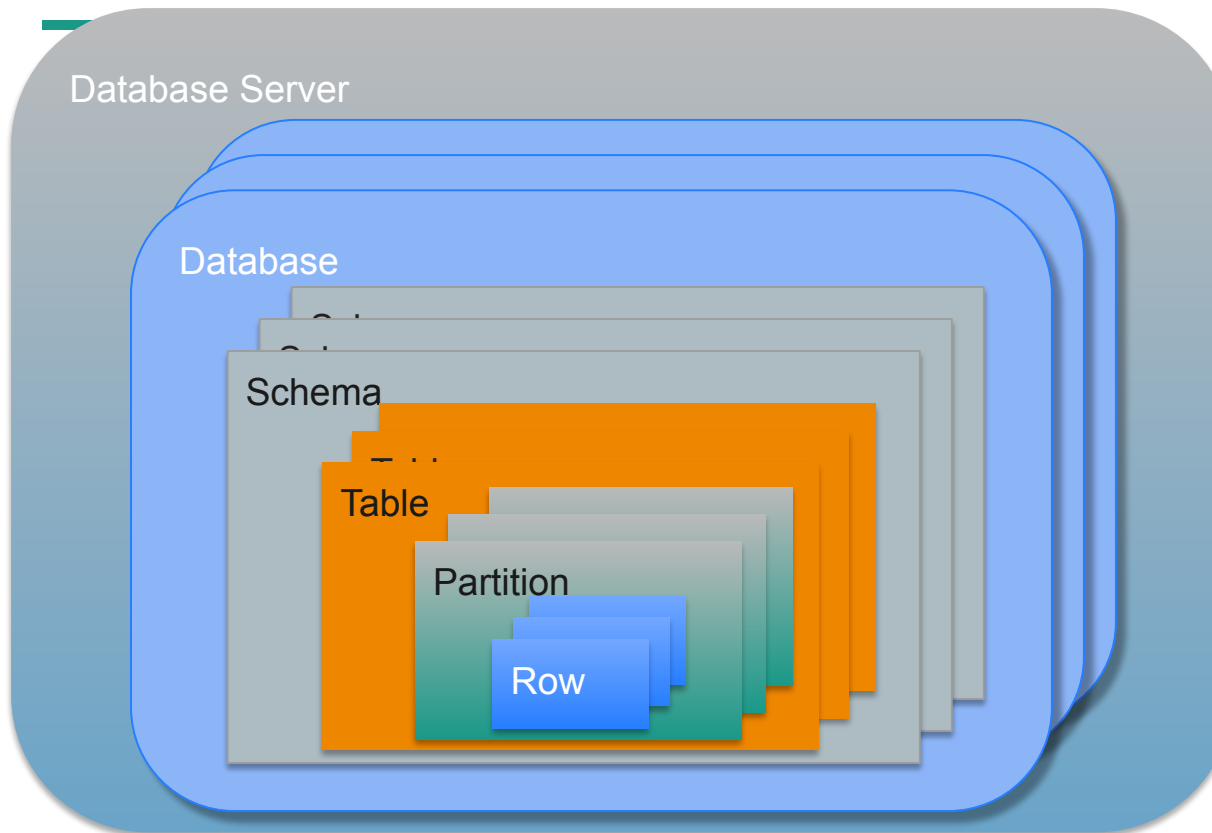


Try it out

- 1) Add products to myproduct table (up to 5 total)
- 2) Add several customers (max 5)
- 3) Add at least 5 orders for different products and customers. Orders dating in February 2025
- 4) Create a query that displays the orders with
 - Order id
 - Order date (ascending)
 - Customer name in format 'last_name, first_name'
 - Product name
 - Product price
 - Order quantity
 - Extended price (quantity * price)

Organizing Data - Schemas and Entity Relationship Diagrams

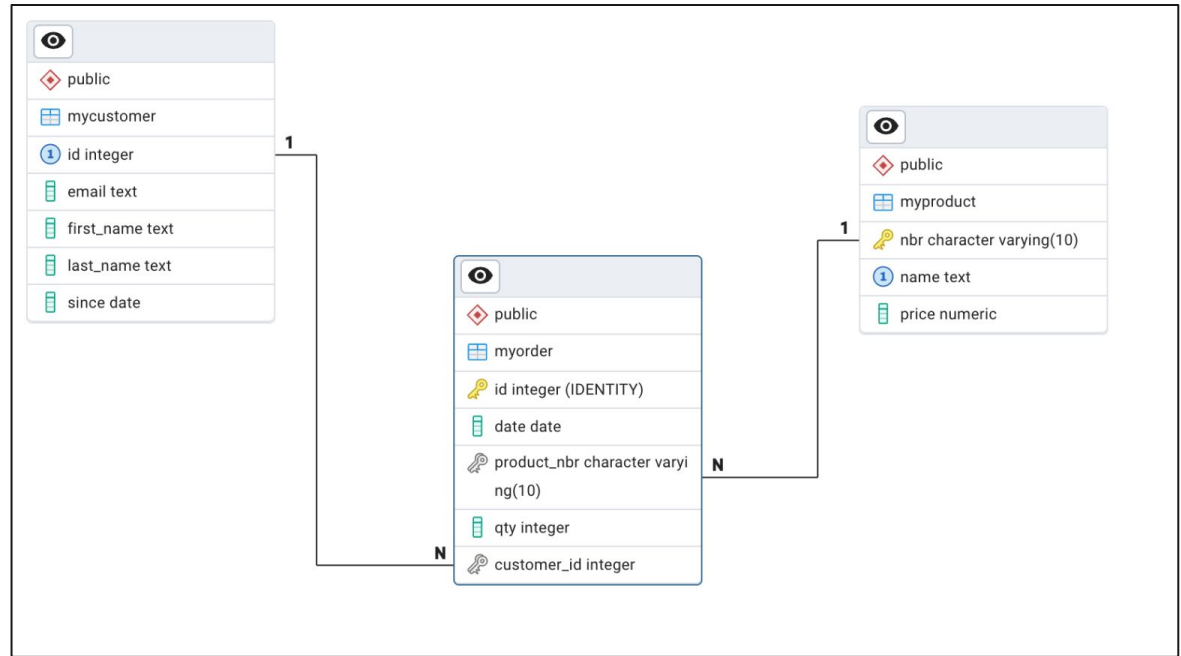
Hierarchy of Postgres and SQL



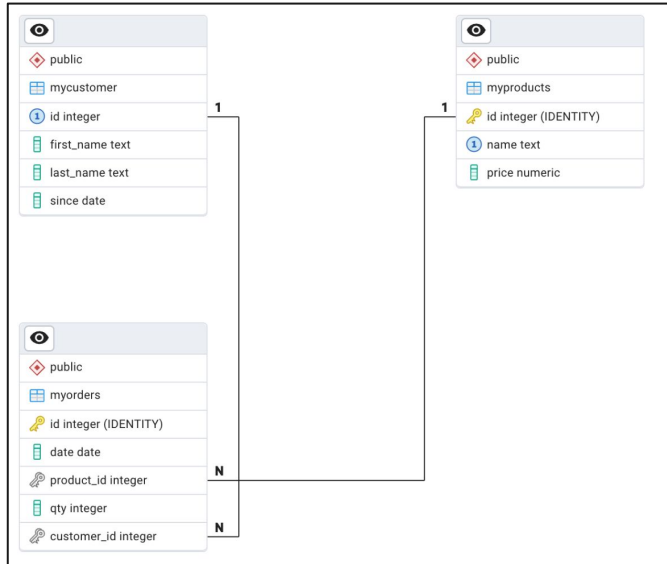
Schemas

Graphical view of the schema

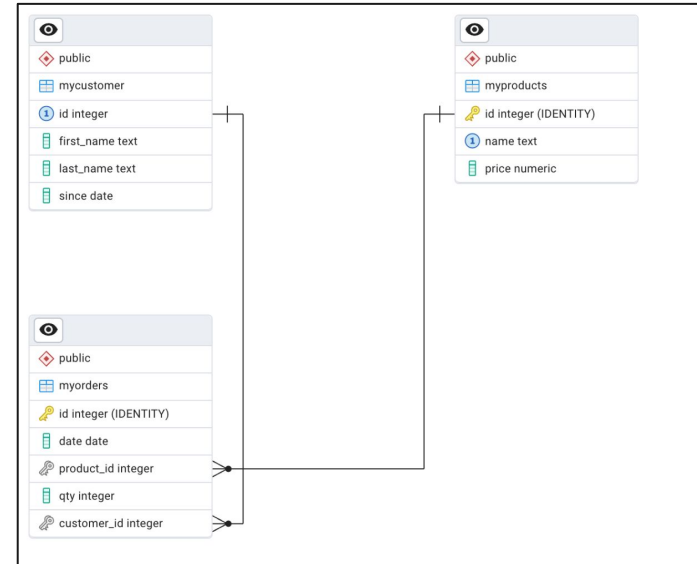
- Tables
- Columns
- Constraints
- Relationships
 - 1:1
 - 1:N
 - N:M



Two Notations



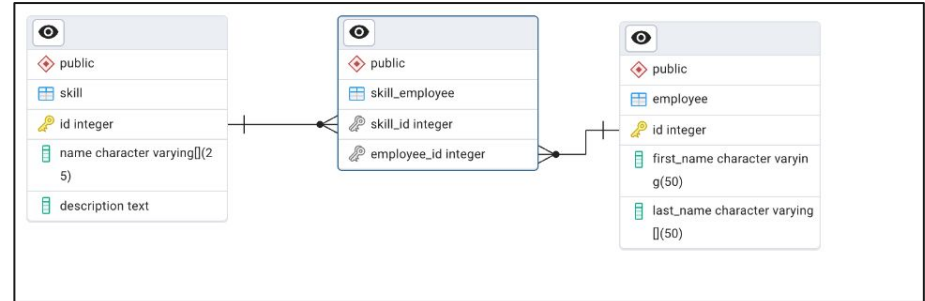
Chen Notation



Crow Foot Notation

Use ERD Diagramming to create simple database

- Make sure you are connected to database DLH2
- Use pgAdmin ERD tool to define tables
 - Employees
 - Skills
 - Employees that have skill sets
- Use ERD tool to generate SQL
- Execute the generated SQL
- Inspect the results using the tree browser



Remove the **BEGIN;** and **END;** statement at the end of the generated SQL before executing

Data Normalization



Normalization and Avoiding Redundancy

- Data Normalization, a.k.a. 1NF/2NF,3NF is key to relational database design
- Data Normalization concepts do not exist in non-relational databases
 - Make sure every column only has one data element of the same type
 - Avoid redundancy (same data in multiple places)
 - Eliminate unintentional dependencies



Database Normal Forms

- **1st Normal Form (1NF):** Tables do not include other tables; every column has one data element of the same type
- **2nd Normal Form (2NF):** All data elements depend on the whole primary key
- **3rd Normal Form (3NF):** No internal dependencies between data elements

Other (higher) normal forms exist, but are rarely applied in practice



Example

| Title | Author | Author Nationality | Format | Price | Subject | Pages | Thickness | Publisher | Publisher Country | Genre ID | Genre Name |
|--|------------------|--------------------|-----------|-------|-----------------------|-------|-----------|-----------|-------------------|----------|------------|
| Beginning MySQL Database Design and Optimization | Chad Russel
1 | American | Hardcover | 49.99 | MySQL Database Design | 520 | Thick | Apress | USA | 1 | Tutorial |



Composite Primary Key Candidates

Subject has multiple values in one column

| <u>Title</u> | Author | Author Nationality | <u>Format</u> | Price | Subject | Pages | Thickness | Publisher | Publisher Country | Genre ID | Genre Name |
|--|--------------|--------------------|---------------|-------|-----------------------|-------|-----------|-----------|-------------------|----------|------------|
| Beginning MySQL Database Design and Optimization | Chad Russell | American | Hardcover | 49.99 | MySQL Database Design | 520 | Thick | Apress | USA | 1 | Tutorial |



Normal Form 1: Every column has one value

| <u>Title</u> | Author | Author Nationality | <u>Format</u> | Price | Pages | Thickness | Publisher | Publisher Country | Genre ID | Genre Name |
|--|--------------|--------------------|---------------|-------|-------|-----------|-----------|-------------------|----------|------------|
| Beginning MySQL Database Design and Optimization | Chad Russell | American | Hardcover | 49.99 | 520 | Thick | Apress | USA | 1 | Tutorial |

| Title | Subject name |
|--|--------------|
| Beginning MySQL Database Design and Optimization | MySQL |
| Beginning MySQL Database Design and Optimization | Database |
| Beginning MySQL Database Design and Optimization | Design |

Expended Example: 2nd Normal Form

Price depends on Title and Format – everything else depends on title

| <u>Title</u> | <u>Format</u> | Author | Author Nationality | Price | Pages | Thickness | Publisher | Publisher Country | Genre ID | Genre Name |
|---|---------------|--------------|--------------------|-------|-------|-----------|----------------|-------------------|----------|-----------------|
| Beginning MySQL Database Design and Optimization | Hardcover | Chad Russell | American | 49.99 | 520 | Thick | Apress | USA | 1 | Tutorial |
| Beginning MySQL Database Design and Optimization | E-book | Chad Russell | American | 22.34 | 520 | Thick | Apress | USA | 1 | Tutorial |
| The Relational Model for Database Management: Version 2 | E-book | E.F.Codd | British | 13.88 | 538 | Thick | Addison-Wesley | USA | 2 | Popular science |
| The Relational Model for Database Management: Version 2 | Paperback | E.F.Codd | British | 39.99 | 538 | Thick | Addison-Wesley | USA | 2 | Popular science |



2nd Normal Form - clean up key dependencies

| <u>Title</u> | Author | Author Nationality | Pages | Thickness | Publisher | Publisher Country | Genre ID | Genre Name |
|---|--------------|--------------------|-------|-----------|----------------|-------------------|----------|-----------------|
| Beginning MySQL Database Design and Optimization | Chad Russell | American | 520 | Thick | Apress | USA | 1 | Tutorial |
| The Relational Model for Database Management: Version 2 | E.F.Codd | British | 538 | Thick | Addison-Wesley | USA | 2 | Popular science |

| <u>Title</u> | <u>Format</u> | <u>Price</u> |
|---|---------------|--------------|
| Beginning MySQL Database Design and Optimization | Hardcover | 49.99 |
| Beginning MySQL Database Design and Optimization | E-book | 22.34 |
| The Relational Model for Database Management: Version 2 | E-book | 13.88 |
| The Relational Model for Database Management: Version 2 | Paperback | 39.99 |

3rd Normal Form - remove dependencies

Depends on
author

Depends on
publisher

| <u>Title</u> | Author | Author Nationality | Pages | Thickness | Publisher | Publisher Country | Genre ID | Genre Name |
|---|--------------|--------------------|-------|-----------|----------------|-------------------|----------|-----------------|
| Beginning MySQL Database Design and Optimization | Chad Russell | American | 520 | Thick | Apress | USA | 1 | Tutorial |
| The Relational Model for Database Management: Version 2 | E.F.Codd | British | 538 | Thick | Addison-Wesley | USA | 2 | Popular science |

Depends on
genre id



3rd Normal Form

| <u>Title</u> | <u>Author</u> | <u>Pages</u> | <u>Thickness</u> | <u>Publisher</u> | <u>Genre ID</u> |
|---|---------------|--------------|------------------|------------------|-----------------|
| Beginning MySQL Database Design and Optimization | Chad Russell | 520 | Thick | Apress | 1 |
| The Relational Model for Database Management: Version 2 | E.F.Codd | 538 | Thick | Addison-Wesley | 2 |

| <u>Title</u> | <u>Format</u> | <u>Price</u> |
|---|---------------|--------------|
| Beginning MySQL Database Design and Optimization | Hardcover | 49.99 |
| Beginning MySQL Database Design and Optimization | E-book | 22.34 |
| The Relational Model for Database Management: Version 2 | E-book | 13.88 |
| The Relational Model for Database Management: Version 2 | Paperback | 39.99 |

| <u>Publisher</u> | <u>Country</u> |
|------------------|----------------|
| Apress | USA |
| Addison-Wesley | USA |

| <u>Genre ID</u> | <u>Name</u> |
|-----------------|-----------------|
| 1 | Tutorial |
| 2 | Popular science |

| <u>Author</u> | <u>Author Nationality</u> |
|---------------|---------------------------|
| Chad Russell | American |
| E.F.Codd | British |



Normalization Matters!

- Avoids managing the same data in multiple places
- Avoids strange deletion situations, e.g., book genres disappear when no book currently refers to them
- Makes sure there is always a way to refer to data in a unique way
- Avoids designing the process into the data
 - The initial design was good to add new books
 - But you could not add publishers or genres that had no books currently
 - Do you design for 'Add new Books' or 'Find all books of a certain genre and author'

Day 3

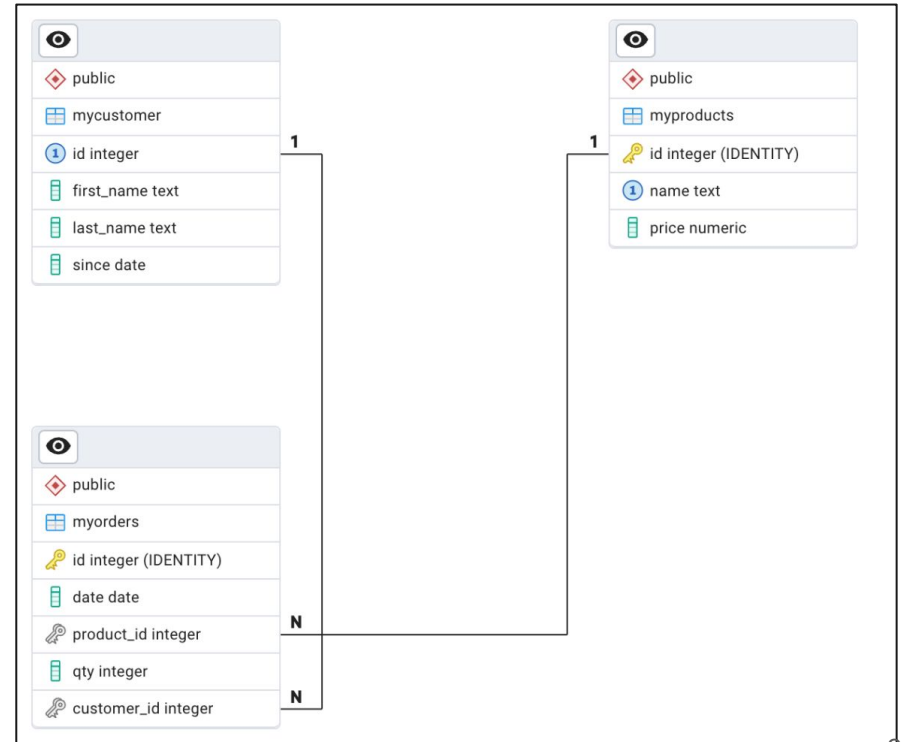
- Review of Day 2
- Queries revisited (joins, aggregates)
- Indexes and the Explain Tool

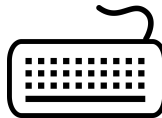
Joins

'JOIN ... ON ...' connects Two or more tables

Types of joins:

- (INNER) JOIN
- LEFT/RIGHT (OUTER) JOIN
- FULL (OUTER) JOIN
- CROSS JOIN

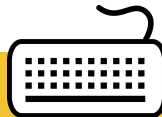




Customers who ordered something

```
SELECT * FROM myorder
JOIN mycustomer
ON myorder.customer_id = mycustomer.id;
```

| id | date | product_nbr | qty | customer_id | id | email | first_name | last_name | since |
|----|------------|-------------|-----|-------------|----|----------------------|------------|-----------|------------|
| 1 | 2025-02-02 | chisel_004 | 9 | 1 | 1 | marc@marclinster.com | Marc | Linster | 2025-01-01 |



```

SELECT
    o.id AS order_id,
    o.date AS order_date,
    o.qty AS order_quantity,
    c.first_name || ' ' || c.last_name AS customer_name
FROM myorder o
    JOIN mycustomer c
    ON o.customer_id = c.id;

```

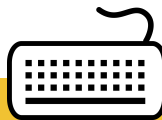
```

order_id | order_date | order_quantity | customer_name
-----+-----+-----+-----
        1 | 2025-02-02 |                | Marc Linster

```

- N.B.:
 - Explicit JOIN/ON improves readability
 - Using table alias (o,c) improves readability and avoids errors

How about the customers without orders?



LEFT OUTER JOIN

```
SELECT * FROM mycustomer c
      LEFT OUTER JOIN myorder o
      ON c.id = o.customer_id;
```

| id | first_name | last_name | since | id | date | product_id | qty | customer_id |
|----|------------|-----------|------------|----|------------|------------|-----|-------------|
| 1 | Marc | Linster | 2025-01-01 | 1 | 2025-02-02 | 3 | 3 | 1 |
| 2 | Jeff | Mueller | 2025-01-02 | | | | | |
| 3 | Jean | Bintner | 2025-01-03 | | | | | |

LEFT (OUTER) JOIN:

- All records from the first (left) table
- Joined with second (right) table if available ; show NULL (empty) columns otherwise
- 'OUTER' is optional



Explaining Left Join

mycustomer (left of join)

| id | email | first_ | last_ | since |
|----|----------------------|--------|---------|----------|
| 1 | marc@marclinster.com | Marc | Linster | 1/1/2025 |
| 2 | jmu@gmail.lu | Jeff | Mueller | 1/2/2025 |
| 3 | bini@hotmail.lu | Jean | Bintner | 1/3/2025 |

myorder (right of join)

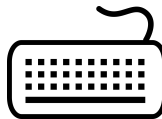
| id | date | product_nbr | qty | customer_id |
|----|----------|-------------|-----|-------------|
| 1 | 2/2/2025 | chisel_004 | 9 | 1 |

In simple join:

- align customers and orders using `mycustomer.id = myorder.customer_id`
- List only the customers that have a matching order, and the orders that have a matching customer

In left join:

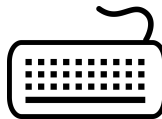
- align customers and orders using `mycustomer.id = myorder.customer_id`
- List all the customers (left) and if there is a matching order, list it too



Joining more tables: Customer with orders and product details

```
SELECT * FROM mycustomer c
        JOIN myorder o ON c.id = o.customer_id
        JOIN myproduct p ON o.product_id = p.id;
```

| id | first_name | last_name | since | id | date | product_id | qty | customer_id | id | name | price |
|----|------------|-----------|------------|----|------------|------------|-----|-------------|----|--------|-------|
| 1 | Marc | Linster | 2025-01-01 | 1 | 2025-02-02 | 3 | 3 | 1 | 3 | pliers | 2 |



All customers, with orders and products (if they ordered anything)

```
SELECT * FROM mycustomer c
      LEFT JOIN myorder o ON c.id = o.customer_id
      LEFT JOIN myproduct p ON o.product_nbr = p.nbr;
```

| id | first_name | last_name | since | id | date | product_id | qty | customer_id | id | name | price |
|----|------------|-----------|------------|----|------------|------------|-----|-------------|----|--------|-------|
| 1 | Marc | Linster | 2025-01-01 | 1 | 2025-02-02 | 3 | 3 | 1 | 3 | pliers | 2 |
| 2 | Jeff | Mueller | 2025-01-02 | | | | | | | | |
| 3 | Jean | Bintner | 2025-01-03 | | | | | | | | |

(3 rows)

- See the importance of meaningful alias
- 'id' appears three times.
- customer_id, order_id, product_id would improve readability and reduce errors

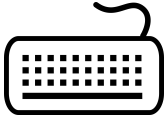


Try It Out

- Find the customers who didn't order anything.
 - Show only the name and since when they are customers
- Find the products that were never ordered

```
SELECT DISTINCT(p.*) FROM myproduct p
      LEFT JOIN myorder o ON o.product_nbr = p. nbr
      WHERE o.id IS NULL
```

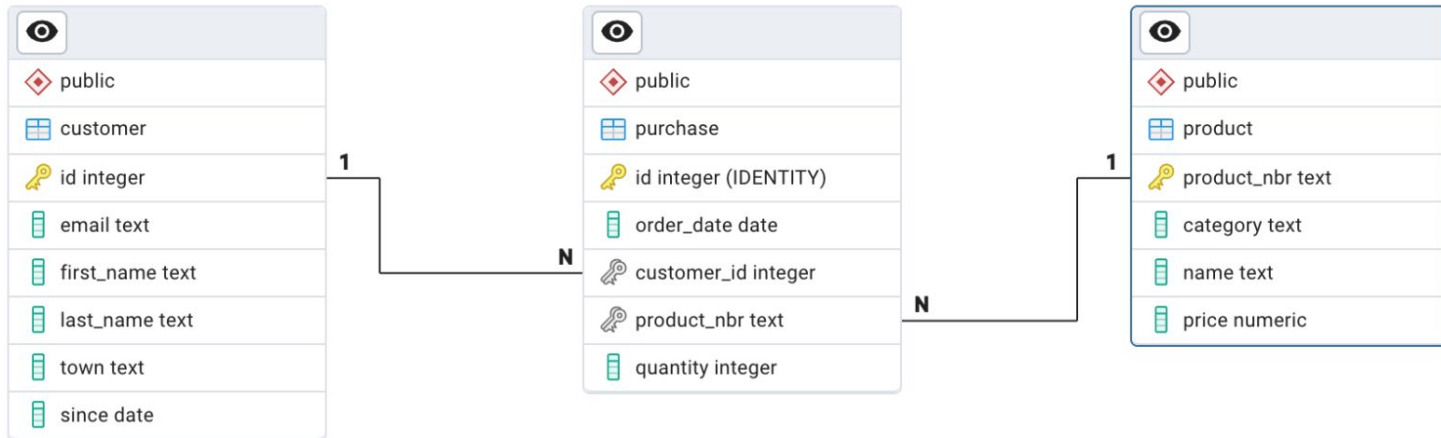
Views



Explore the data set

- Connect to database DLH3
- Create the ERD Diagram
- Explore the tables
 - How many customers?
 - How many customers from Koerich?
 - List all the customers from Koerich that became a customer in June 2025.
Order by date asc
 - How many orders were placed by customers from Differdange?
 - Which products did customers from Koerich order?
 - Which product categories did customers from Koerich order?

ERD Diagram



CREATE VIEW

Views are named queries that are stored in the database server

```
CREATE VIEW customer_purchase_view AS
SELECT
    c.id AS customer_id,
    c.first_name || ' ' || c.last_name AS
customer_name,
    c.town,
    p.product_nbr,
    pr.name AS product_name,
    pr.category as product_category,
    p.quantity,
    p.id AS purchase_id,
    p.order_date,
    (p.quantity * pr.price) AS total_price
FROM customer c
JOIN purchase p ON c.id = p.customer_id
JOIN product pr ON p.product_nbr = pr.product_nbr;
```


pgAdmin 4

Object Explorer

- Servers (1)
 - postgres17
 - Databases (3)
 - dlh1
 - Catalogs
 - Extensions
 - Languages
 - Schemas
 - dlh3
 - Catalogs
 - Extensions
 - Languages
 - Schemas (1)
 - public
 - Sequences
 - Tables
 - Views (1)
 - customer_purchase_view
 - postgres
 - Catalogs
 - Extensions
 - Languages

dlh3/postgres@postgres17

customer_purchase_view

General Definition Code Security SQL

```
1 SELECT c.id AS customer_id,
2       (c.first_name || ' '::text) || c.last_name AS customer_name,
3       c.town,
4       p.product_nbr,
5       pr.name AS product_name,
6       pr.category AS product_category,
7       p.quantity,
8       p.id AS purchase_id,
9       p.order_date,
10      p.quantity::numeric * pr.price AS total_price
11 FROM customer c
12      JOIN purchase p ON c.id = p.customer_id
13      JOIN product pr ON p.product_nbr = pr.product_nbr;
```

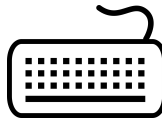
Total rows: Screenshot LF Ln 1, Col 1



Nota Bene

- VIEWS run the underlying queries every time they are accessed.
- VIEWS make it easy to share the same definitions within a team - i.e., every analyst uses the same definitions
- VIEWS are convenient and reduce code, but they do not significantly increase performance
-
- MATERIALIZED VIEWS are like VIEWS, but they create a table behind the scenes
- MATERIALIZED VIEWS pre-calculate the results
- MATERIALIZED VIEWS are
 - Much, much faster
 - Can (must) have their own indexes
 - Must be refreshed when the underlying data changes (In Postgres; Oracle has an incremental refresh)
 - Can be refreshed concurrently (no locking - needs a primary key) or non-concurrently (with locking - no PK needed)
- MATERIALIZED VIEWS are not in scope of the introductory course

Aggregates



Explore the data set

- How many customers?
- How many customers from Koerich?
- List all the customers from Koerich that became a customer in June 2025. Order by date asc
- How many orders were placed by customers from Differdange?
- Which products did customers from Koerich order?
- Which product categories did customers from Koerich order?



Questions that require aggregates

For example

- How many customers do we have in every town?
- Which town has the most customers?
- How much did the top 10 customers buy?



SQL Aggregates

- GROUP BY, e.g., group the customers by location
 - GROUP BY ... HAVING allows to limit the groups
- SUM, AVG, MIN, MAX, e.g, the total/ average/ minimum/ maximum within a group

} After the WHERE clause

} Part of the SELECT clause



Aggregates

```
SELECT group_by_element_1, group_by_element_2, ..., aggregate_1, aggregate_2, ...  
      FROM table1  
      JOIN table2 ON ....  
      WHERE condition  
      GROUP BY group_by_element_1, group_by_element_2, ...
```

```
SELECT town, COUNT(id)  
FROM customer  
GROUP BY TOWN  
ORDER BY town ASC;
```



Aggregates

```
SELECT town, COUNT(id) customer_count FROM customer
GROUP BY TOWN
ORDER BY town ASC;
```

Single group - one aggregate (COUNT)

```
SELECT town, COUNT(id) AS customer_count FROM customer
WHERE since >= '2024-01-01' AND since < '2024-12-31'
GROUP BY TOWN
HAVING COUNT(id) > 1;
```

Single group - one aggregate; WHERE clause + limit of the groups

```
SELECT town, COUNT(id) AS customer_count FROM customer
WHERE since >= '2024-01-01' AND since < '2024-12-31'
GROUP BY TOWN
HAVING COUNT(id) > 1
ORDER BY customer_count DESC;
```

Single group - one aggregate; WHERE clause + limit of the groups + ORDER BY for groups



Aggregates

```
SELECT
    town,
    EXTRACT (YEAR FROM since) AS since_year,
    COUNT(id) AS customer_count FROM customer
GROUP BY town, since_year
HAVING COUNT(id) > 1
ORDER BY town ASC, since_year ASC;
```

```
SELECT
    town,
    EXTRACT (YEAR FROM since) AS since_year,
    COUNT(id) AS customer_count FROM customer
WHERE TOWN <> 'ESCH'
GROUP BY town, since_year
HAVING COUNT(id) > 1
ORDER BY town ASC, since_year ASC;
```

Use the 'HAVING' clause to remove groups with small customer counts

Use the 'HAVING' clause to remove groups with small customer counts
+
Use WHERE clause to filter on data set

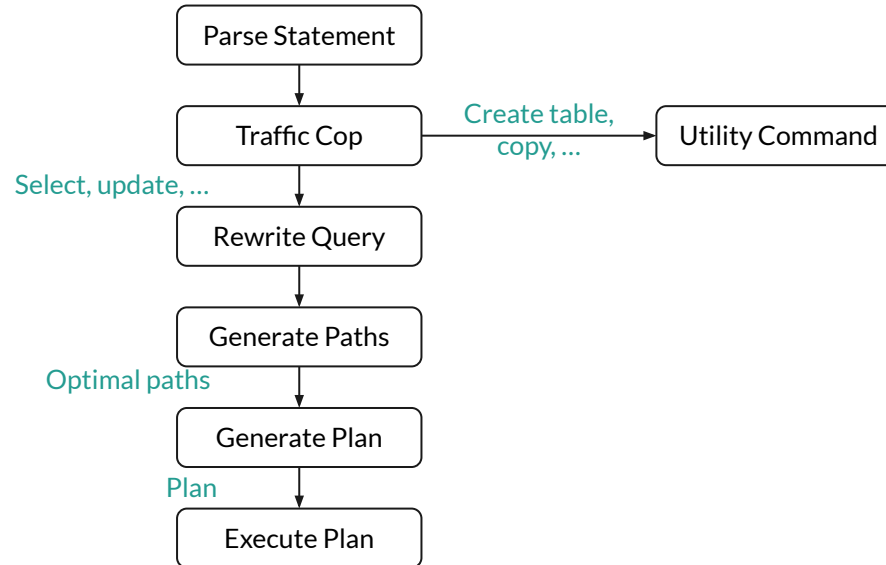


Exercises

- How many food items did we sell in 2024
 - Join purchases and products
 - Filter by category food
 - Count(*)
- How many things did the top 10 customers buy?
 - Join the customer table with the purchase table
 - Group by customer and count the quantities
 - LIMIT 10
- How much did the top 10 customers spend every month?
 - Take a look at the view customer_purchase_view
 - Group by customer_name and make an aggregate of the total price
 - Order by price
 - LIMIT 10

Indexes

How does Postgres process a query?



See <https://momjian.us/main/writings/pgsql/internalpics.pdf>



How does a database find data?

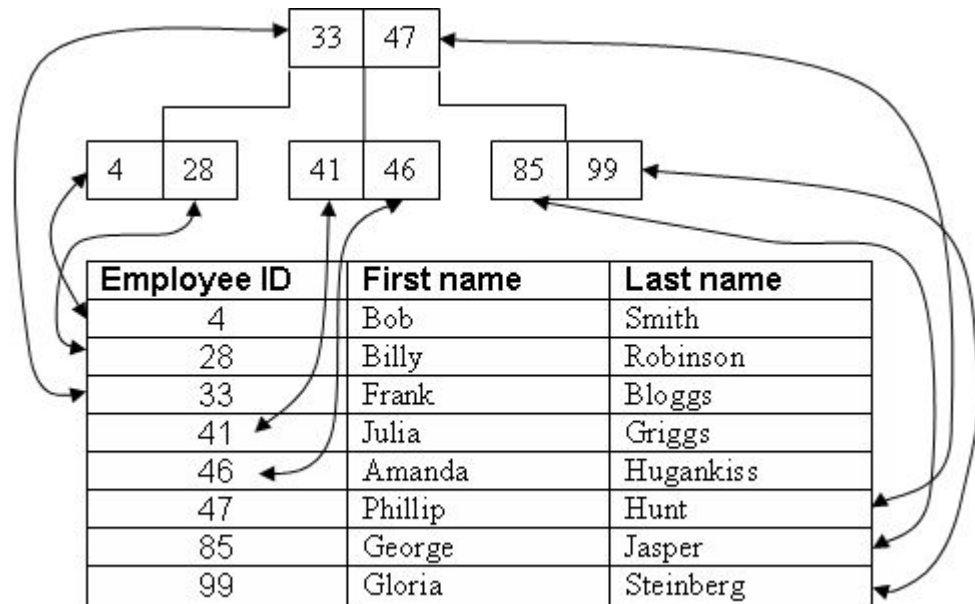
- Every table is stored in 8k data blocks on disk, in files that are 1GB (or less)
- Example: `SELECT * from customer WHERE last_name = 'Voigt';`
- Without Index
 - Sequential scan of all data blocks for that table
 - If block is not in memory (shared buffers), get it from disk and load it into memory
 - “Sequential Scan”
- With Index
 - Load the index into memory (usually much smaller than data table)
 - Ask the index which blocks have the data
 - Load only those blocks
 - “Index Scan”

Many types of Indexes



- **Balanced Tree (B-Tree)**
 - Every database has it. The most common and used for 90%+ of all cases. The default in Postgres
 - Ideal for looking up unique values and maintaining unique indexes
 - High concurrency implementation - very fast
- **Block-Range Index (BRIN)**
 - Ideal for very large datasets that are organized according to the index
- **Generalized Inverted Index (GIN)**
 - Best for indexing values with many keys or values, e.g., text documents, JSON, multi-dimensional data, arrays
 - Ideal for columns containing many duplicate
- **Hash Index**
 - Equality, non-equality lookups; no range lookups
- **Generalized Search Tree (GiST)**
 - Used for GIS, key-value store
- **Space-Partitioned Generalized Search Tree (SP-GIST)**
 - Specialized index

Very Fast Access to Data



<https://en.wikipedia.org/wiki/Database>

Writing Good Queries



Summary: Keys to Writing Good Queries

1. Limit the number of rows that are returned
2. Limit the number of columns that are returned
3. Leverage indexes where available
4. Define aliases in the SELECT clause using 'AS' to avoid duplicate column names
5. Use the aliases in the WHERE and GROUP clause to avoid typos and shorten the code
6. Define highly selective WHERE clauses, especially when using JOINS
7. Avoid CROSS JOINS (data set explodes!)
8. Use VIEWS to standardize complex, often repeated queries (e.g., sales_transaction combined with sales_transaction_line)
9. Use snake_case names for tables and columns
10. Study EXPLAIN plans to understand where complexity occurs

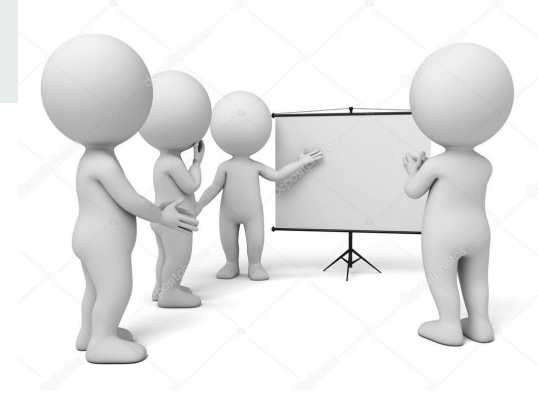


SQL Order of Execution

1. **FROM/JOIN:** Specifies the tables from which to retrieve data.
2. **WHERE:** Filters the rows that meet the condition before grouping.
3. **GROUP BY:** Groups rows that share a property.
4. **HAVING:** Filters groups based on conditions, applied after grouping.
5. **SELECT:** Specifies the columns to retrieve or calculate.
6. **DISTINCT:** Removes duplicate rows from the result set.
7. **ORDER BY:** Sorts the result set by specified columns.
8. **LIMIT:** Specifies the maximum number of rows to return.
9. **OFFSET:** Specifies how many rows to skip before starting to return rows.

A sample project

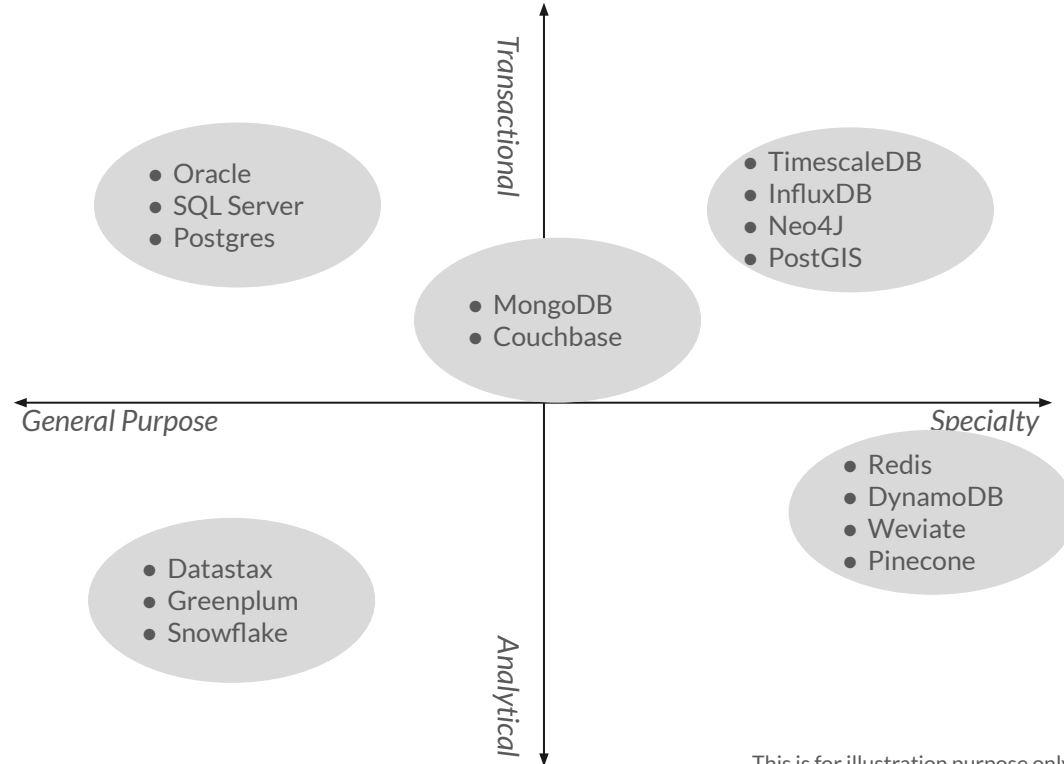
Project



- Build a database (DLHX) for a company with departments, employees and managers working in departments, projects, employee assignment to projects, and hours recorded for the employees working on the project
- Simple queries
 - Who worked on which project?
 - Who had the most hours total last month?
 - What were the total hours last month for each project?
 - Did we have any projects with 0 hours?
 - What is the reporting hierarchy in the company?

Recap

Database Landscape



This is for illustration purpose only
There are many, many databases and many ways to classify them

Table Basics

- Rows = records (a.k.a. tuples in Postgres)
- Columns = data elements
- Example: mycustomer table
 - ID = a way to uniquely identify a customer ⇒ usually a large number
 - Email = the email address, a text field, sometimes limited in length
 - First name ⇒ a text field, sometimes limited in length
 - Last name ⇒ a text field, sometimes limited in length
 - Since => a date field

```
CREATE TABLE mycustomer (
    id INTEGER,
    email TEXT,
    first_name TEXT,
    last_name TEXT,
    since DATE
);
```

```
postgres=# SELECT * FROM mycustomer;
```

| id | email | first_name | last_name | since |
|----|----------------------|------------|-----------|------------|
| 1 | marc@marclinster.com | Marc | Linstner | 2025-01-01 |
| 2 | jmu@gmail.lu | Jeff | Mueller | 2025-01-02 |
| 3 | bini@hotmail.lu | Jean | Bintner | 2025-01-03 |



Query types

- SELECT: List data from tables that meet certain conditions
- UPDATE: Change data in records that meet certain conditions
- INSERT: Add records
- DELETE: Remove Records



Building Blocks of the simple SELECT Query

- **SELECT**
 - Columns to be selected, maybe using AS to provide a meaningful name
 - For example `last_name AS ln`
- **FROM**
 - Table(s) from which to select. Can also use AS to provide a new name
 - For example `customer AS c`
- **WHERE**
 - Filter clauses to select a subset of records
 - Can use the new names from the AS clause
 - For example `c.last_name LIKE 'Li%'`
- **ORDER BY**
 - Organizes the output of the SELECT clause
 - Can use the new names from the AS clause
 - For example: `c.last_name ASC, since DESC`

Alias (AS)

- improves readability
- Only available in the context of this query
- Key word 'AS' is optional

More complex building blocks for SELECT queries (JOIN, GROUP BY, HAVING, GROUPING, WITH, ...) will be introduced later



Postgres Operators in the WHERE Clause

| | |
|---------|--|
| = | Equal to |
| < | Less than |
| > | Greater than |
| <= | Less than or equal to |
| >= | Greater than or equal to |
| <> | Not equal to |
| != | Not equal to |
| LIKE | Check if a value matches a pattern (case sensitive) |
| ILIKE | Check if a value matches a pattern (case insensitive) |
| AND | Logical AND |
| OR | Logical OR |
| IN | Check if a value is between a range of values |
| BETWEEN | Check if a value is between a range of values |
| IS NULL | Check if a value is NULL |
| NOT | Makes a negative result e.g. NOT LIKE, NOT IN, NOT BETWEEN |

Postgres functions in the SELECT clause

Some of the key string functions

| | | |
|------------------------|---|---|
| <code> </code> | String append | <code>'a' '_' 'b' → a_b</code> |
| <code>lpad/rpad</code> | Create a padded string | <code>rpad('hi', 5, 'xy') → hixyx</code> |
| <code>substring</code> | Create a substring | <code>substring('Thomas' from 2 for 3) → hom</code> |
| <code>::string</code> | Cast a number (and other types) as string | <code>1::TEXT '_' 2::TEXT → 1_2</code> |

Many, many more in the Postgres documentation: <https://www.postgresql.org/docs/current/functions-string.html>



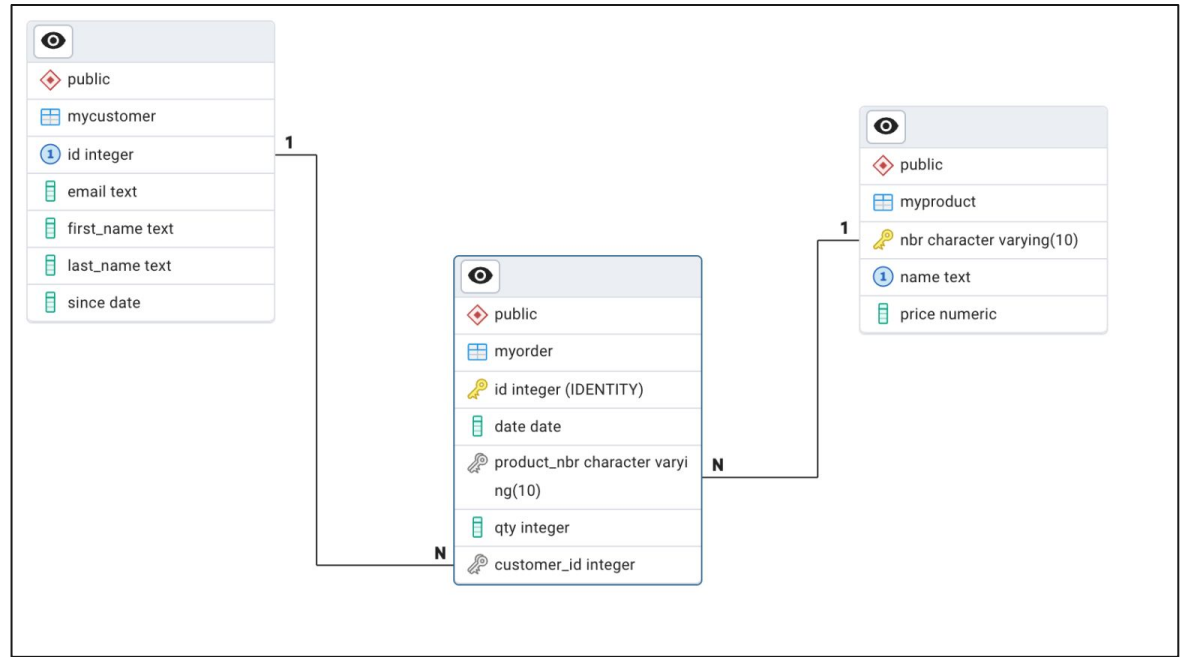
Constraints and Relationships

- **Check Constraints:** The value in the column has to meet certain criteria, e.g., price > 0
- **Not-Null Constraints:** The column needs to have a value
- **Unique Constraints:** No two records have the same value
- **Primary Keys:** like the Unique Constraint, but excludes NULL and always creates an index
 - ⇒ A primary key can also be a combination of columns
- **Foreign Keys:** the data needs to be a valid reference to another table
 - You can assign a 'trigger' to handle deletions

Schemas

Graphical view of the schema

- Tables
- Columns
- Constraints
- Relationships
 - 1:1
 - 1:N
 - N:M





3rd Normal Form

| <u>Title</u> | <u>Author</u> | <u>Pages</u> | <u>Thickness</u> | <u>Publisher</u> | <u>Genre ID</u> |
|---|---------------|--------------|------------------|------------------|-----------------|
| Beginning MySQL Database Design and Optimization | Chad Russell | 520 | Thick | Apress | 1 |
| The Relational Model for Database Management: Version 2 | E.F.Codd | 538 | Thick | Addison-Wesley | 2 |

| <u>Title</u> | <u>Format</u> | <u>Price</u> |
|---|---------------|--------------|
| Beginning MySQL Database Design and Optimization | Hardcover | 49.99 |
| Beginning MySQL Database Design and Optimization | E-book | 22.34 |
| The Relational Model for Database Management: Version 2 | E-book | 13.88 |
| The Relational Model for Database Management: Version 2 | Paperback | 39.99 |

| <u>Publisher</u> | <u>Country</u> |
|------------------|----------------|
| Apress | USA |
| Addison-Wesley | USA |

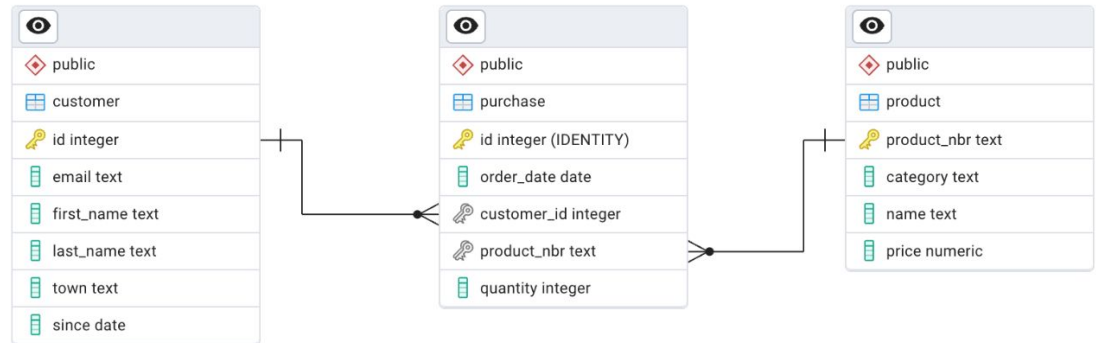
| <u>Genre ID</u> | <u>Name</u> |
|-----------------|-----------------|
| 1 | Tutorial |
| 2 | Popular science |

| <u>Author</u> | <u>Author Nationality</u> |
|---------------|---------------------------|
| Chad Russell | American |
| E.F.Codd | British |

'JOIN ... ON ...' connects Two or more tables

Types of joins:

- (INNER) JOIN
- LEFT/RIGHT (OUTER) JOIN
- FULL (OUTER) JOIN
- CROSS JOIN



CREATE VIEW

Views are named queries that are stored in the database server

```
CREATE OR REPLACE VIEW mycustomer_with_purchase_view AS
SELECT
    c.id AS customer_id,
    FORMAT ('%s %s', c.first_name, c.last_name)
        AS customer_name,
    c.since AS customer_since,
    o.id AS order_number,
    o.date AS order_date,
    p.name AS product_name,
    p.price AS product_unit_price,
    p.price * o.qty AS extended_price
FROM mycustomer c
    LEFT JOIN myorder o ON c.id = o.customer_id
    LEFT JOIN myproduct p ON o.product_nbr = p.nbr;

SELECT * FROM mycustomer_with_purchase_view;
```

Try \d and \d+ mycustomer_with_purchase_view

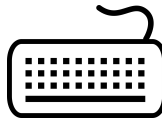


SQL Aggregates

- GROUP BY, e.g., group the customers by location
 - GROUP BY ... HAVING allows to limit the groups
- SUM, AVG, MIN, MAX, e.g, the total/ average/ minimum/ maximum within a group

} After the WHERE clause

} Part of the SELECT clause



How many customers by town?

```
SELECT town, COUNT(id) customer_count FROM customer
      GROUP BY TOWN
      ORDER BY town ASC;
```

```
SELECT
    town,
    EXTRACT (YEAR FROM since) AS since_year,
    COUNT(id) AS customer_count FROM customer
WHERE TOWN <> 'ESCH'
GROUP BY town, since_year
HAVING COUNT(id) > 1
ORDER BY town ASC, since_year ASC;
```

Many types of Indexes



- **Balanced Tree (B-Tree)**
 - Every database has it. The most common and used for 90%+ of all cases. The default in Postgres
 - Ideal for looking up unique values and maintaining unique indexes
 - High concurrency implementation - very fast
- **Block-Range Index (BRIN)**
 - Ideal for very large datasets that are organized according to the index
- **Generalized Inverted Index (GIN)**
 - Best for indexing values with many keys or values, e.g., text documents, JSON, multi-dimensional data, arrays
 - Ideal for columns containing many duplicate
- **Hash Index**
 - Equality, non-equality lookups; no range lookups
- **Generalized Search Tree (GiST)**
 - Used for GIS, key-value store
- **Space-Partitioned Generalized Search Tree (SP-GIST)**
 - Specialized index



Summary: Keys to Writing Good Queries

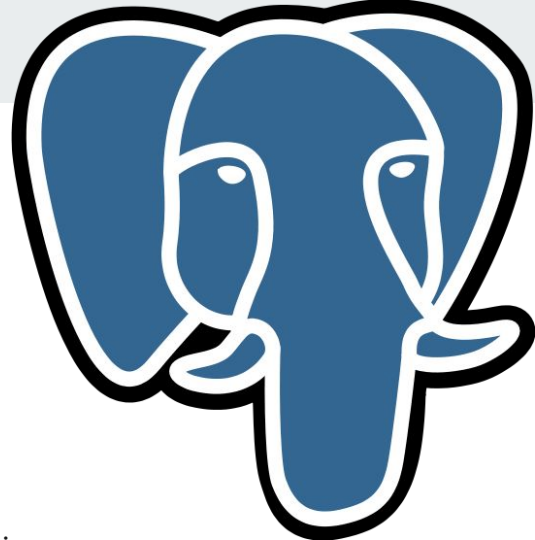
1. Limit the number of rows that are returned
2. Limit the number of columns that are returned
3. Leverage indexes where available
4. Define aliases in the SELECT clause using 'AS' to avoid duplicate column names
5. Use the aliases in the WHERE and GROUP clause to avoid typos and shorten the code
6. Define highly selective WHERE clauses, especially when using JOINS
7. Avoid CROSS JOINS (data set explodes!)
8. Use VIEWS to standardize complex, often repeated queries (e.g., sales_transaction combined with sales_transaction_line)
9. Use snake_case names for tables and columns
10. Study EXPLAIN plans to understand where complexity occurs



SQL Order of Execution

1. **FROM/JOIN:** Specifies the tables from which to retrieve data.
2. **WHERE:** Filters the rows that meet the condition before grouping.
3. **GROUP BY:** Groups rows that share a property.
4. **HAVING:** Filters groups based on conditions, applied after grouping.
5. **SELECT:** Specifies the columns to retrieve or calculate.
6. **DISTINCT:** Removes duplicate rows from the result set.
7. **ORDER BY:** Sorts the result set by specified columns.
8. **LIMIT:** Specifies the maximum number of rows to return.
9. **OFFSET:** Specifies how many rows to skip before starting to return rows.

Resources



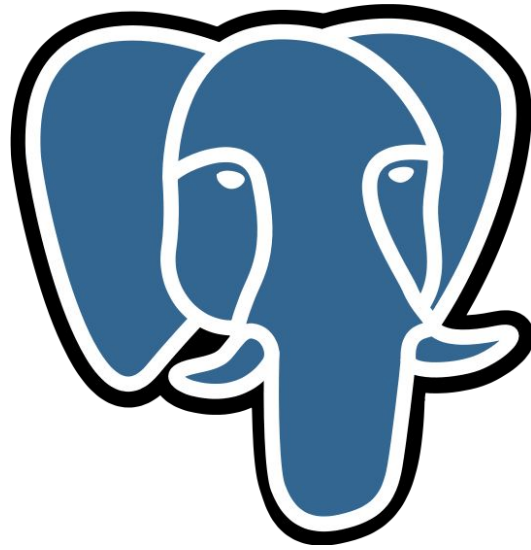
Resources

Books:

- PostgreSQL 16 Administration Cookbook; Gianni Ciolli, Boris Mejias, Jimmy Angelakos, Vibhor Kumar, Simon Riggs, 2023
- PostgreSQL Up and Running; Regina Obe and Leo Hsu, 2020
- Introduction to PostgreSQL for the data professional; Ryan Booz and Grant Fritchey 2025 (coming from SQL Server)
- Mastering PostgreSQL 17: Elevate your database skills with advanced deployment, optimization, and security strategies 6th ed. Hans-Jürgen Schönig, 2023

Blogs & online:

- Postgres documentation: <https://www.postgresql.org/docs/current/index.html>
- Neon's Postgres Tutorial: <https://neon.tech/postgresql/tutorial>
- Bruce Momjian's blog about SQL and Postgres: <https://momjian.us/main/presentations/sql.html>
- Bruce Momjian's blog about Postgres performance: <https://momjian.us/main/presentations/performance.html>
- Laetitia Avrot's excellent blog about DBA topics: <https://mydbanotebook.org/>
- Vibhor Kumar's blog: <https://vibhorkumar.wordpress.com/>
- Stormatics' blog: <https://stormatics.tech/our-blogs>
- pgAdmin Website (with videos): <https://www.pgadmin.org/>



Use Postgres - Get Stuff Done!